# INDEX


| Sr, No | Practical | Page No. | Date | Signature |
|---|---|---|---|---|
| 1 | Implementing Substitution and Transposition Cipher: Design and implement algorithms to encrypt and decrypt messages using classical substitution and transposition techniques. | 3 | 27\06\2024 | |
| 2 | RSA Encryption and Decryption: Implementing the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations. | 5 | 04\07\2024 | |
| 3 | Message Authentication Codes: Implement algorithm to generate and verify message authentication codes(MACs) for ensuring data integrity and authenticity of digitally signed message. | 10 | 11\07\2024 | |
| 4 | Digital Signature: Implement Digital Signature Algorithms such as RSA-based signature, and verify the integrity and authenticity of digitally signed messages. | 11 | 25\07\2024 | |
| 5 | Key Exchange using Diffie-Hellman: Implement the Diffie-Hellman key exchange algorithm to security exchange keys between two entities over an insecure network. | 13 | 01\08\2024 | |
| 6 | IP Security(IPsec) Configuration: Configure IPsec on network devices to provide secure communication and protect against unauthorized access and attacks | 14 | 08\08\2024 | |
| 7 | Web Security with SSL/TLS: Configure and implement secure web communication using SSL/TLS protocols, including certificate management and secure session establishment. | 19 | 29\08\2024 | |
| 8 | Intrusion Detection System: Setup and configure an intrusion detection(IDS) to monitor network traffic and detect potential security breaches or malicious activities. | 21 | 12\09\2024 | |

| | | | | |
|---|---|---|---|---|
| 9 | Malware Analysis and Detection:<br>Analyze and Identity malware samples using antivirus tools, analyze their behavior and develop counter measures to mitigate their impact. | 23 | 19\09\2024 | |
| 10 | Firewall Configuration and Rule-based Filtering: Configure and test firewall rules to control network traffic, filter packets based on specified criteria, and protect network<br>resources from unauthorized access. | 26 | 03\10\2024 | |

# Practical No. 1

**Aim:** To study and implement Substitution Cipher and Transposition Cipher.

**Theory:**

**Substitution Cipher:**

A Substitution Cipher is a type of encryption method where each letter or symbol in the Plain text (original message) is replaced with another letter or symbol according to a fixed system or rule. The key characteristic is that each character in the plaintext maps to a unique character in the ciphertext (encrypted message), maintaining the sequence of the original text but disguising its meaning.

This type of cipher can be categorized into monoalphabetic and polyalphabetic ciphers. In a monoalphabetic cipher, each letter of the plaintext is substituted with a single, consistent letter of the ciphertext. A polyalphabetic cipher uses multiple substitution alphabets that vary according to a key throughout the encryption process, such as in the Vigenere cipher.

**Transposition Cipher:**

A Transposition cipher is a method of encryption where the positions of the characters in the plaintext are rearranged according to a specific system or key, but the characters themselves remain unchanged. Unlike substitution ciphers, which replace characters with others, transposition ciphers keep the original characters but alter their order.

This method works by altering the order of the letters rather than substituting them with other characters. For instance, in a columnar transposition cipher, the plaintext message is written out in rows of a certain fixed length, and then the message is read out column by column according to the sequence specified by a predetermined key. Transposition ciphers keep the letters intact but shuffle their order.

**Input:**

**Substitution Cipher:**

```
class SubstitutionCipher: def  init       (self, key):
self.key = key

def encrypt(self, plaintext): encrypted_text = ""

for char in plaintext: if char.isalpha():
if char.islower():
encrypted_text += self.key[ord(char) - ord('a')].lower() else:
encrypted_text += self.key[ord(char) - ord('A')].upper()
else:
encrypted_text += char

return encrypted_text

def decrypt(self, ciphertext): decrypted_text = ""

for char in ciphertext: if char.isalpha():
if char.islower():
decrypted_text += chr(self.key.lower().index(char) + ord('a')) else:
decrypted_text += chr(self.key.upper().index(char) + ord('A'))
else:
decrypted_text += char

return decrypted_text

key = "welkfghlserigherispagjuoiphetopsasd" sub_cipher = SubstitutionCipher(key) message = "Hello
World!"
encrypted_message = sub_cipher.encrypt(message) decrypted_message =
sub_cipher.decrypt(encrypted_message)

print("Original: ", message) print("Encrypted: ", encrypted_message)

print("Decrypted: ", decrypted_message)
```

**Output:**

```
PS C:\Users\admin\Desktop\New folder> & "C:/Program Files/Python312/python.exe" "c:/Users/admin/Desktop/New fol
der/ins1.py"
Original:  Hello World!
Encrypted:  Lfiie Uesik!
Decrypted:  Cellb Wbild!
PS C:\Users\admin\Desktop\New folder>
```

**Transposition Cipher**

**Input:**

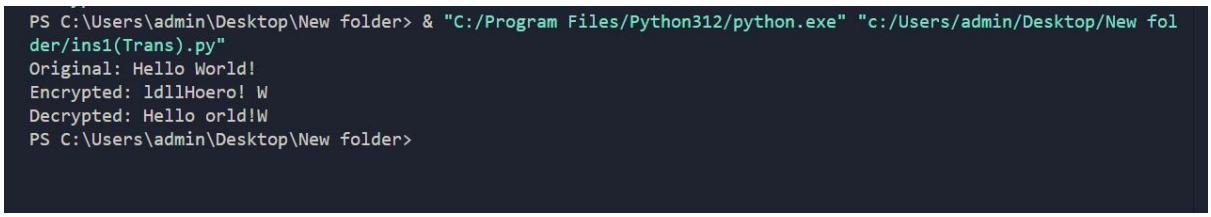
```
class TranspositionCipher: def        init
        (self, key):
self.key = key


def encrypt(self, plaintext): num_columns =
len(self.key)
num_rows = len(plaintext) // num_columns +
(1 if len(plaintext) % num_columns != 0 else
0)
matrix = [['] * num_columns for _ in
range(num_rows)] for i, char in
enumerate(plaintext):
matrix[i // num_columns][i % num_columns]
= char encrypted_text = ""
for col in self.key: col_index = int(col) - 1
encrypted_text +=
''.join(matrix[row][col_index] for row in
range(num_rows))
return encrypted_text
def decrypt(self, ciphertext): num_columns =
len(self.key)
num_rows = len(ciphertext) // num_columns +
(1 if len(ciphertext) % num_columns != 0 else
0)
matrix = [['] * num_columns for _ in
range(num_rows)] char_index = 0
for col in self.key: col_index = int(col) - 1
for row in range(num_rows):
if char_index < len(ciphertext):
matrix[row][col_index] =
ciphertext[char_index] char_index += 1
decrypted_text = ""
for i in range(num_rows): decrypted_text +=
''.join(matrix[i])
return decrypted_text
# Example usage:
key = "4312567"
trans_cipher = TranspositionCipher(key)
message = "Hello World!"
```

```
encrypted_message =
trans_cipher.encrypt(message)
decrypted_message =
trans_cipher.decrypt(encrypted_message)


print("Original:", message) print("Encrypted:",
encrypted_message) print("Decrypted:",
decrypted_message)
```

**Output:**



# Practical No. 2

**Aim:** To study and implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

**Theory:**

**RSA Algorithm:**

The RSA comes from three inventors: Ronald Rivest, Adi Shamir, Leonard Adleman - the three MIT mathematicians.

RSA is widely used:

Key exchange Digital

Signature

Encryption of small blocks of data

For encryption purpose, RSA uses varying size encryption block and varying key size where each user generates a public / private key pair.

Here's a breakdown of the RSA algorithm in three main steps: deriving the RSA key pair, encryption, and decryption, followed by an example for better understanding.

**Step 1:** Deriving the RSA Key Pair

1. Choose Two Prime Numbers: Select two large prime numbers p and q.
2. Compute n : Multiply p and q to get n . n = p

   x q
3. Compute z :

   z = (p-1) x (q-1)
4. Choose e such that e < n and it has no common factors with z. That means, e and z are relatively prime.

Choose d such that ed - 1 exactly divisible by z. That means, ed mod z = 1.

5. Form Key Pair:

- Public Key: (n, e)
- Private Key: (n, d)

**Step 2:** RSA Encryption

1. To encrypt bit pattern (plaintext message) m, compute C = m^e mod n where c is the remainder where

2. Encrypt the Message: C

= m^e mod n

**Step 3:** RSA Decryption

1. To Decrypt received bit pattern (ciphertext message) c, compute m = c^d mod n where m is the remainder when c^d is divided ny n.

**Input:**

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP import binascii
keyPair = RSA.generate(1024)

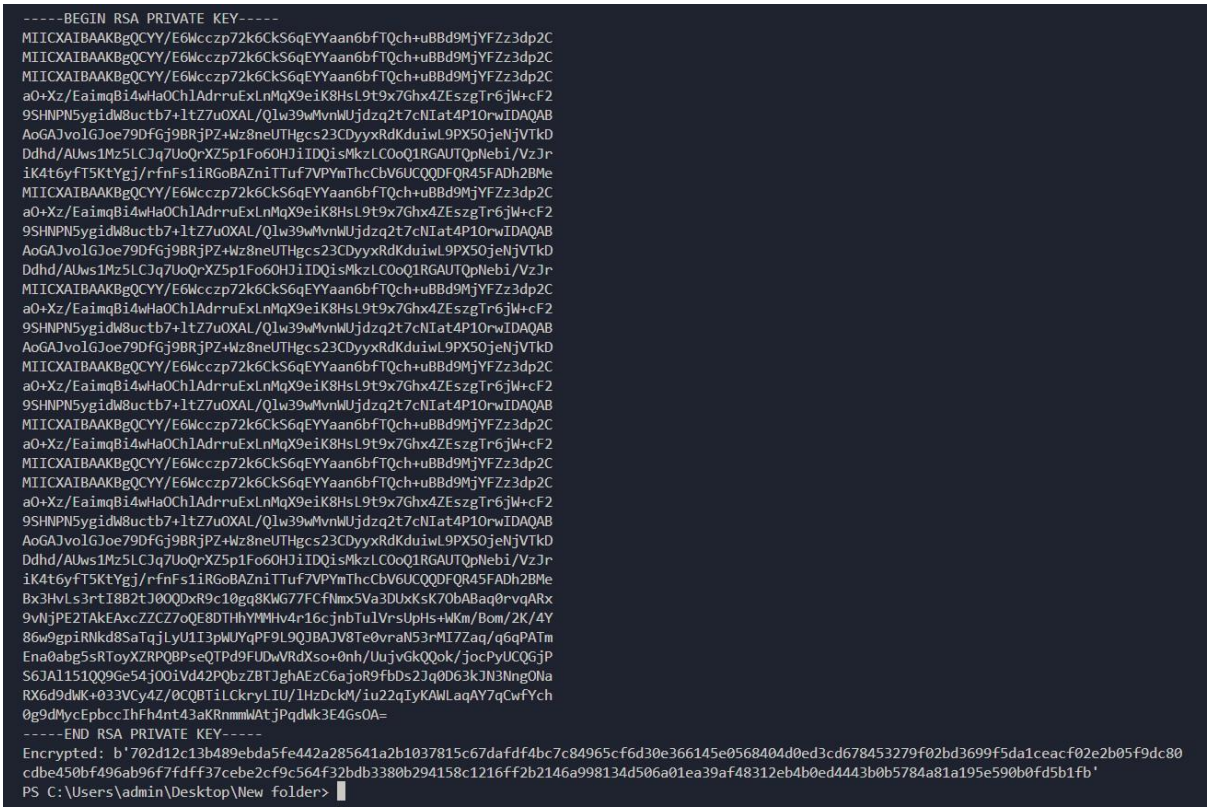
pubkey = keyPair.publickey()
print(f"Public key: (n={hex(pubkey.n)} , e={hex(pubkey.e)} )") pubkeyPEM =
pubkey.export_key() print(pubkeyPEM.decode('ascii'))

print(f"Private key: (n={hex(pubkey.n)} , d={hex(keyPair.d)} )") privkeyPEM =
keyPair.export_key() print(privkeyPEM.decode('ascii'))

msg = b'IsmileAcademy'
encryptor = PKCS1_OAEP.new(pubkey) encrypted =
encryptor.encrypt(msg) print("Encrypted:"
binascii.hexlify(encrypted)
```

```
PS C:\Users\admin\Desktop\New folder> pip install pycryptodome
>>
Defaulting to user installation because normal site-packages is not writeable
Collecting pycryptodome
  Downloading pycryptodome-3.20.0-cp35-abi3-win_amd64.whl.metadata (3.4 kB)
Downloading pycryptodome-3.20.0-cp35-abi3-win_amd64.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 15.9 MB/s eta 0:00:00
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.20.0
PS C:\Users\admin\Desktop\New folder> & "C:/Program Files/Python312/python.exe" "c:/Users/admin/Desktop/New folder/ins2(RSA).py"
Public key: (n=0x9863f13a59c733a7bda4e82912eaa11861a6a7e9b7d341c87eb8105df4c8d8159cf7769d8268ef97cff11a8a6a818b8c0768e0a194076baee1312e732a5
fd7a22bc1ec2fdb7dc7b1a1c78644b33813afa8d6f9c176f521cd3cde72822756f2e72d6fbfa5b59eee39700bfd0970dfdc0cbe75948ddceadadedc3486ade0fd4eaf , e=0x
10001 )
-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQCYY/E6Wcczp72k6CkS6qEYYaan
6bfTQch+uBBd9MjYFZz3dp2CaO+Xz/EaimqBi4wHaOChlAdrruExLnMqX9eiK8Hs
L9t9x7Ghx4ZEszgTr6jW+cF29SHNPN5ygidW8uctb7+ltZ7uOXAL/Qlw39wMvnWU
jdzq2t7cNIat4P1OrwIDAQAB
-----END PUBLIC KEY-----
Private key: (n=0x9863f13a59c733a7bda4e82912eaa11861a6a7e9b7d341c87eb8105df4c8d8159cf7769d8268ef97cff11a8a6a818b8c0768e0a194076baee1312e732a
5fd7a22bc1ec2fdb7dc7b1a1c78644b33813afa8d6f9c176f521cd3cde72822756f2e72d6fbfa5b59eee39700bfd0970dfdc0cbe75948ddceadadedc3486ade0fd4eaf , d=0
x26fa25189a1eefd0df1a3f414633d9f96cfc9de5131e072cdb7083cb2c5174a76e8b02fd3d7e4e8de363553903 0dd85dfc0530b35333e4b089abb52842b5d9e69d45a3a3872
62203422b0c9332c23a84354460144d0a4d79b8bf57326b88ae2deb27d3e4ab58823feb7e716cd62446a010199e24d3b9fed53d899385c09b57a5 )
```

**Output:**


-----BEGIN RSA PRIVATE KEY-----
MIICXAIBAAKBgQCYY/E6Wcczp72k6CkS6qEYYaan6bfTQch+uBBd9MjYFZz3dp2C
MIICXAIBAAKBgQCYY/E6Wcczp72k6CkS6qEYYaan6bfTQch+uBBd9MjYFZz3dp2C
MIICXAIBAAKBgQCYY/E6Wcczp72k6CkS6qEYYaan6bfTQch+uBBd9MjYFZz3dp2C
aO+Xz/EaimqBi4wHaOChlAdrruExLnMqX9eiK8HsL9t9x7Ghx4ZEszgTr6jW+cF2
9SHNPN5ygidW8uctb7+ltZ7uOXAL/Qlw39wMvnWUjdzq2t7cNIat4P1OrwIDAQAB
AoGAJvolGJoe79DfGj9BRjPZ+Wz8neUTHgcs23CDyyxRdKduiwL9PX5OjeNjVTkD
Ddhd/AUws1Mz5LCJq7UoQrXZ5p1Fo6OHJiIDQisMkzLCOoQ1RGAUTQpNebi/VzJr
iK4t6yfT5KtYgj/rfnFs1iRGoBAZniTTuf7VPYmThcCbV6UCQQDFQR45FADh2BMe
MIICXAIBAAKBgQCYY/E6Wcczp72k6CkS6qEYYaan6bfTQch+uBBd9MjYFZz3dp2C
aO+Xz/EaimqBi4wHaOChlAdrruExLnMqX9eiK8HsL9t9x7Ghx4ZEszgTr6jW+cF2
9SHNPN5ygidW8uctb7+ltZ7uOXAL/Qlw39wMvnWUjdzq2t7cNIat4P1OrwIDAQAB
AoGAJvolGJoe79DfGj9BRjPZ+Wz8neUTHgcs23CDyyxRdKduiwL9PX5OjeNjVTkD
Ddhd/AUws1Mz5LCJq7UoQrXZ5p1Fo6OHJiIDQisMkzLCOoQ1RGAUTQpNebi/VzJr
MIICXAIBAAKBgQCYY/E6Wcczp72k6CkS6qEYYaan6bfTQch+uBBd9MjYFZz3dp2C
aO+Xz/EaimqBi4wHaOChlAdrruExLnMqX9eiK8HsL9t9x7Ghx4ZEszgTr6jW+cF2
9SHNPN5ygidW8uctb7+ltZ7uOXAL/Qlw39wMvnWUjdzq2t7cNIat4P1OrwIDAQAB
AoGAJvolGJoe79DfGj9BRjPZ+Wz8neUTHgcs23CDyyxRdKduiwL9PX5OjeNjVTkD
MIICXAIBAAKBgQCYY/E6Wcczp72k6CkS6qEYYaan6bfTQch+uBBd9MjYFZz3dp2C
aO+Xz/EaimqBi4wHaOChlAdrruExLnMqX9eiK8HsL9t9x7Ghx4ZEszgTr6jW+cF2
9SHNPN5ygidW8uctb7+ltZ7uOXAL/Qlw39wMvnWUjdzq2t7cNIat4P1OrwIDAQAB
MIICXAIBAAKBgQCYY/E6Wcczp72k6CkS6qEYYaan6bfTQch+uBBd9MjYFZz3dp2C
aO+Xz/EaimqBi4wHaOChlAdrruExLnMqX9eiK8HsL9t9x7Ghx4ZEszgTr6jW+cF2
MIICXAIBAAKBgQCYY/E6Wcczp72k6CkS6qEYYaan6bfTQch+uBBd9MjYFZz3dp2C
MIICXAIBAAKBgQCYY/E6Wcczp72k6CkS6qEYYaan6bfTQch+uBBd9MjYFZz3dp2C
aO+Xz/EaimqBi4wHaOChlAdrruExLnMqX9eiK8HsL9t9x7Ghx4ZEszgTr6jW+cF2
9SHNPN5ygidW8uctb7+ltZ7uOXAL/Qlw39wMvnWUjdzq2t7cNIat4P1OrwIDAQAB
AoGAJvolGJoe79DfGj9BRjPZ+Wz8neUTHgcs23CDyyxRdKduiwL9PX5OjeNjVTkD
Ddhd/AUws1Mz5LCJq7UoQrXZ5p1Fo6OHJiIDQisMkzLCOoQ1RGAUTQpNebi/VzJr
iK4t6yfT5KtYgj/rfnFs1iRGoBAZniTTuf7VPYmThcCbV6UCQQDFQR45FADh2BMe
Bx3HvLs3rtI8B2tJ0OQDxR9c10gq8KWG77FCfNmx5Va3DUxKsK7ObABaq0rvqARx
9vNjPE2TAkEAxcZZCZ7oQE8DTHhYMMHv4r16cjnbTulVrsUpHs+WKm/Bom/2K/4Y
86w9gpiRNkd8SaTqjLyU1I3pWUYqPF9L9QJBAJV8Te0vraN53rMI7Zaq/q6qPATm
Ena0abg5sRToyXZRPQBPseQTPd9FUDwVRdXso+0nh/UujvGkQQok/jocPyUCQGjP
S6JAl151QQ9Ge54jOOiVd42PQbzZBTJghAEzC6ajoR9fbDs2Jq0D63kJN3NngONa
RX6d9dWK+033VCy4Z/0CQBTiLCkryLIU/lHzDckM/iu22qIyKAWLaqAY7qCwfYch
0g9dMycEpbccIhFh4nt43aKRnmmWAtjPqdWk3E4GsOA=
-----END RSA PRIVATE KEY-----
Encrypted: b'702d12c13b489ebda5fe442a285641a2b1037815c67dafdf4bc7c84965cf6d30e366145e0568404d0ed3cd678453279f02bd3699f5da1ceacf02e2b05f9dc80
cdbe450bf496ab96f7fdff37cebe2cf9c564f32bdb3380b294158c1216ff2b2146a998134d506a01ea39af48312eb4b0ed4443b0b5784a81a195e590b0fd5b1fb'
PS C:\Users\admin\Desktop\New folder>

# Practical No. 3

**Aim:** To study and implement the Message Authentication Code for ensuring the message intergrity and authenticity

**Theory:**

A Message Authentication Code (MAC) is a vital cryptographic tool used to ensure the integrity and authenticity of a message. It works by using a secret key and a MAC algorithm to produce a fixed-size checksum from the input message. This checksum, known as the MAC value, is then sent along with the message.

Upon receiving the message and the MAC value, the recipient, who also has the secret key, recomputes the MAC from the received message and compares it with the received MAC value. If the two match, the recipient can be confident that the message has not been tampered with and that it originated from a sender possessing the secret key.

There are various types of MACs, such as HMAC (which uses hash functions), CBC-MAC, and CMAC (which use block ciphers), each with its own advantages depending on the application.

**Input:**

```
import hashlib
result = hashlib.md5(b'Ismile') result1 = hashlib.md5(b'Esmile') result2 = hashlib.md5(b'Niranjan')
print("The byte equivalent of hash is:", end = "") print(result1.digest())
print("The byte equivalent of hash is:", end = "") print(result2.digest())
```

**Output:**

```
PS C:\Users\admin\Desktop\New folder> & "C:/Program Files/Python312/python.exe" "c:/Users/admin/Desktop/New folder/ins3(Mess Authen Code).py"
The byte equivalent of hash is:b'\x900\x10\xfb\xff\x18\xeb\x17\x1d\xa2(kk\x07aX'
The byte equivalent of hash is:b'T\x0cu\x9b\xa5\xe2\x04\x19@\xff\xf6&\xd6\x00\t\xd1'
PS C:\Users\admin\Desktop\New folder>
```

# Practical No. 4

**Aim:** To study and implement the Digital Signature algorithm

**Theory:**

**Digital Signatures:**

Digital Signatures, encryption and message authentication are all considered to be very useful in implementation of modern cryptography.

A signature is known technique used for public key cryptography based on non - repudiation.

Digital Signature is a mathematical technique used to produce authenticity of digital documents.

Digital Signature assures recipients:

1. Authentication: The message is a sent by the known sender.
2. Non - Repudiation: The sender cannot deny that he has mot sent the message.
3. Integrity: While transmitting the message it has not altered.

Integrity of the message and the original source of the document are ensured by attaching code like signature.

Digital Signatures are known to be public key primitives of message authentications.

**Input:**

```
from Crypto.Signature import PKCS1_v1_5 from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA from Crypto import Random

def generate_signature(private_key, message): key = RSA.importKey(private_key)
hashed_message = SHA256.new(message.encode('utf-8')) signer = PKCS1_v1_5.new(key)
signature = signer.sign(hashed_message) return signature

def verify_signature(public_key, message, signature): key = RSA.importKey(public_key)
hashed_message = SHA256.new(message.encode('utf-8')) verifier = PKCS1_v1_5.new(key)
return verifier.verify(hashed_message, signature)

random_generator = Random.new().read
key_pair = RSA.generate(2048, random_generator) public_key = key_pair.publickey().export_key()
private_key = key_pair.export_key()

# Sign a message
message = "Hello, World!"
signature = generate_signature(private_key, message) print("Generated Signature:", signature)

# Verify the signature
is_valid = verify_signature(public_key, message, signature) print("Signature Verification Result:", is_valid)
```

**Output:**

```
Generated Signature: b"\x06`4{\x1foJ\xbd\xa6\x91\x0f?,\xc3\x94\xfdB\x97\x0c\x88\x87rHF\xa8%\x92M\x18\x92\xfe\xdel\x07\x0ej\x1b/\xa5\xe7z(o\xbc\
xf3\xf9F\xec\xdd\xe2\xd3\x0b\x87\x1b:y\xf0\xb4j\xcc\xff\xd4\x18g\x87\xa8\xc0\x12;\xd2\x99\xe0\xcfR\xc4\xb3\xd2\x131\x82\xc1\xe0\x84\xf1\x02\x8d
\xa3\xcc\xc5\xc6a\xef\xecn\x8b\xe6\xb9b\x0e\x91\xb8aEC\xe3\x82\xb9\x0c\xc0\xf35\x86\xf9o\x8fu%\xff\x8c\xd0\x173'/29q\x8cvQ\x13UVpT\xf8\x11a\xc6
\xb3\x82?OV\xa9\xbd\x00/\x8a\x8f3\x951(J\x17G\x03\xac<\xa5G\xac\x82\xec\xefi\x17\xd4y,\xfeO\xf9\x01\x89\xcca\xdb\x0f\xe2\x86^}o\x94 \xa4-\xb0\x
b8\xd1\xeet)i\x14eW\xd2\xd1\x82\x99\x18\xf5\xec|\x08\x93\x8a\x83\xbc\x94/fI\xa1\xef[+\xb05R\xe0\x1dr\x08\x08\xd6\x06\xc0\x01\xe0\xaf\x7f\x15\xe
8\x18\xb5\x8c\r~\x91\xb0\x1fQ\xa6\xc8\xad\x14\x83\x14X\x92\xa3Z"
Signature Verification Result: True
PS C:\Users\admin\Desktop\New folder>
```

# Practical No. 5

**Aim:** To study and implement the Diffe-Hellman key exchange algorithm for secure exchange of keys between two entities.

**Theory:**

**Diffie-Hellman key:**

The Diffie-Hellman key exchange algorithm is produces a session key.

This algorithm solves the problem of key ageement between sender and reciever or exchange of keys between communication parties.

In this method, cryptographic keys are securely exchanged over a public channel.

This method is used for two parties who are unknown to each other, jointly create a secret key over an insecure medium.

Symmetric key is generated which is agreed by both the parties taking part in communication.

The generated key is used for encryption and decryption mechanism.

**Advantages:**

1. Parties involved in communication do not know each other.
2. Secret key sharing is a safe process.

**Disadvantage:**

1. Asymmetric key exchange not supported.
2. Signing of digital signature is not possible

**Input:**

```
from random import randint
if __name      == '      main    ': P = 23
G = 9
print('The Value of P is:', P) print('The Value of G is:', G)
a = 4
print('Secret Number for Alice is:', a) x = int(pow(G, a, P))
b = 6
print('Secret Number for Bob is:', b) y = int(pow(G, b, P))
ka = int(pow(y, a, P))
kb = int(pow(x, b, P))
print('Secret key for the Alice is:', ka) print('Secret Key for the Bob is:', kb)
```

**Output:**

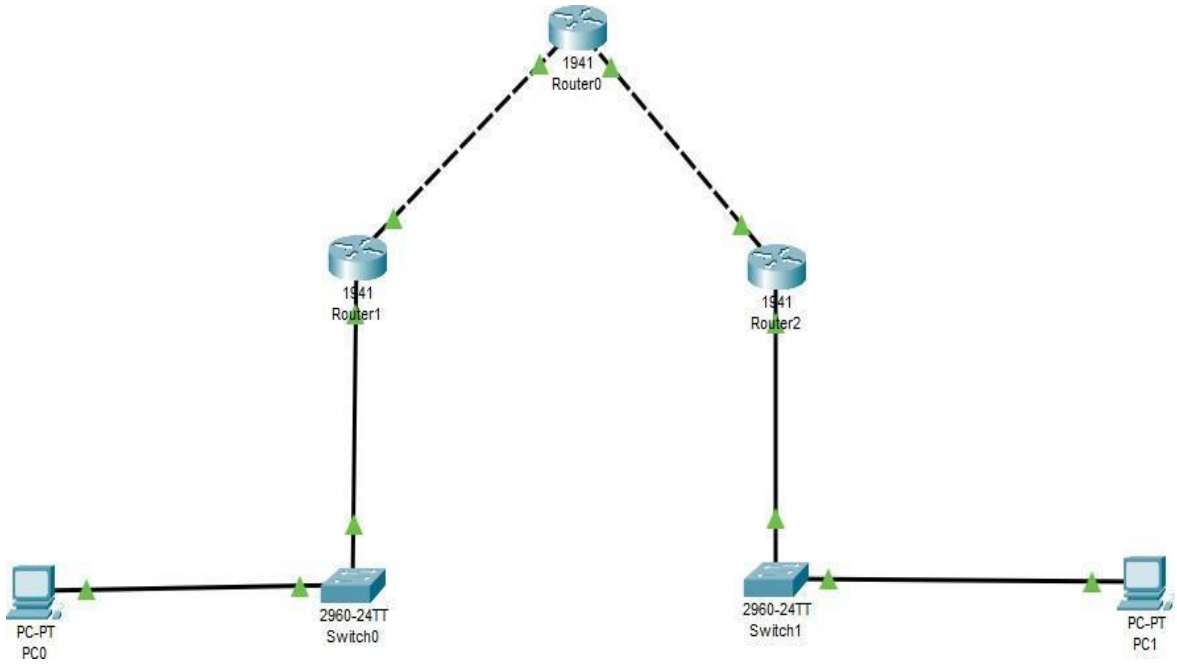


# Practical No. 6

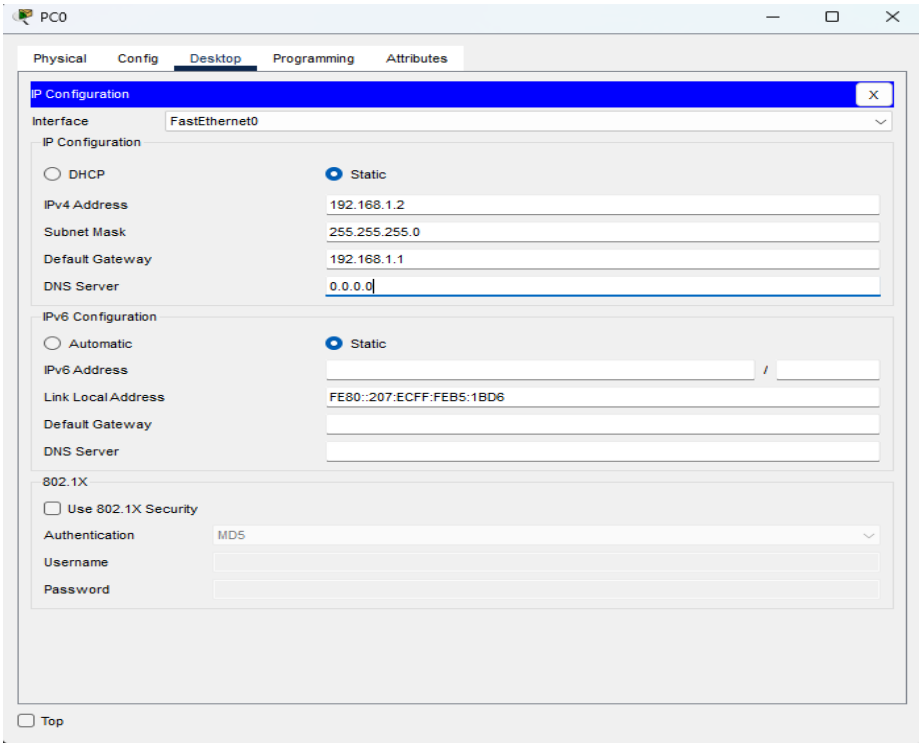
**Aim:** To Configure IPSec on network devices to provide secure communication and protect against unauthorized access and attacks.

**Theory:**

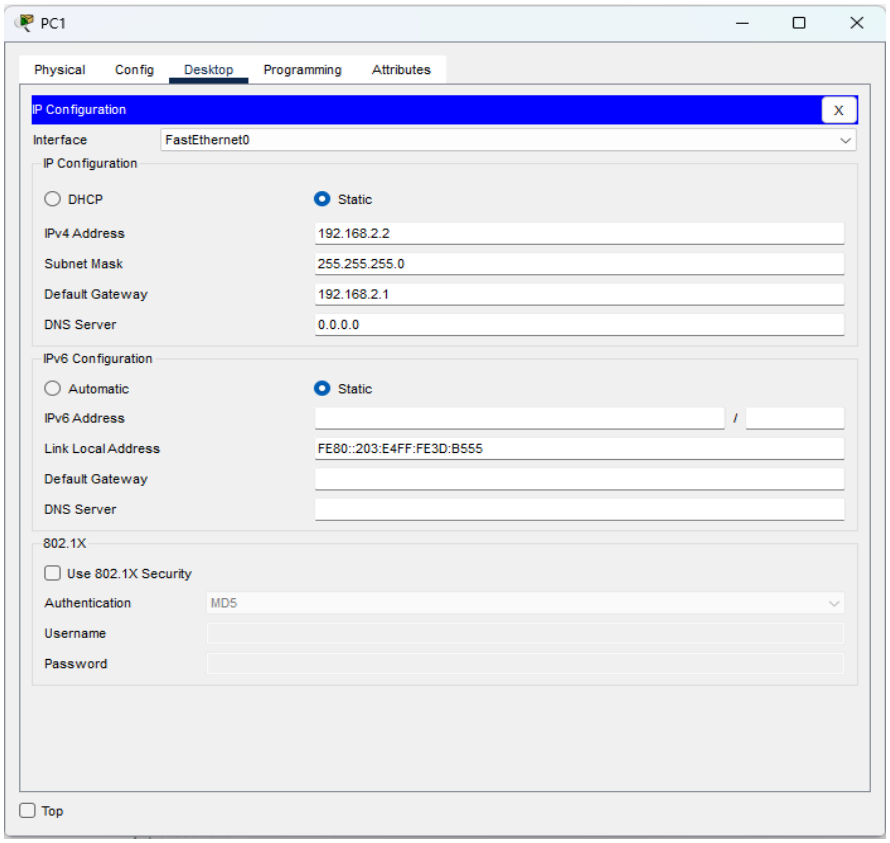
**Input:**




**Configure PC0:**

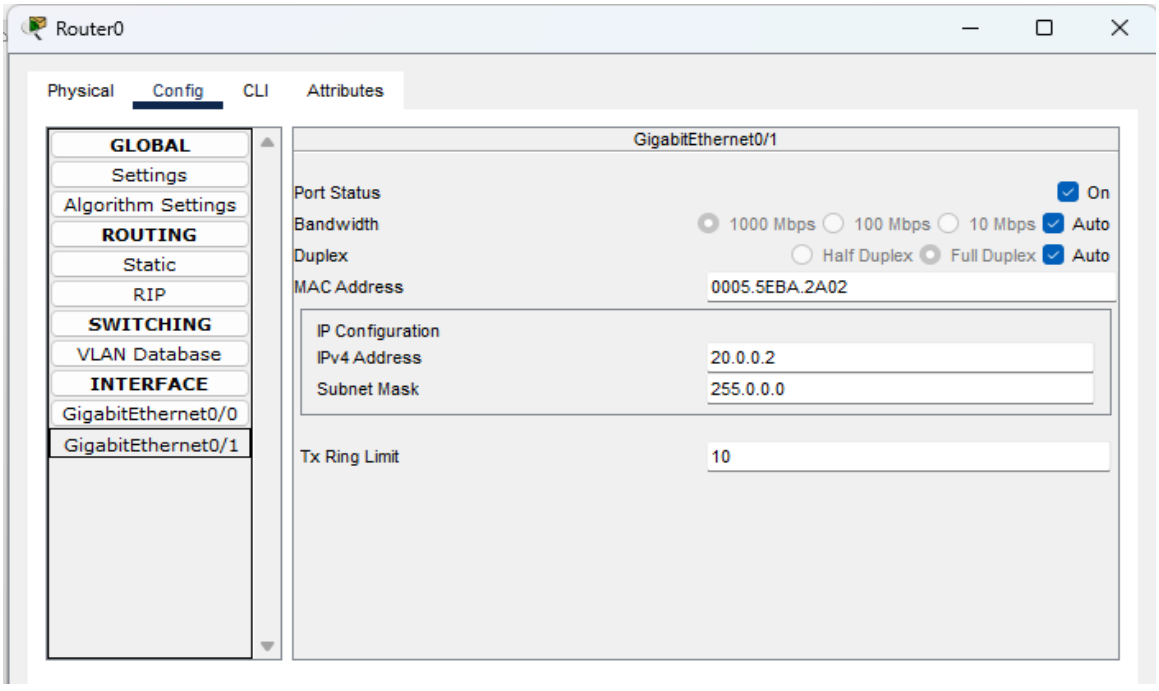


## Configure PC1:



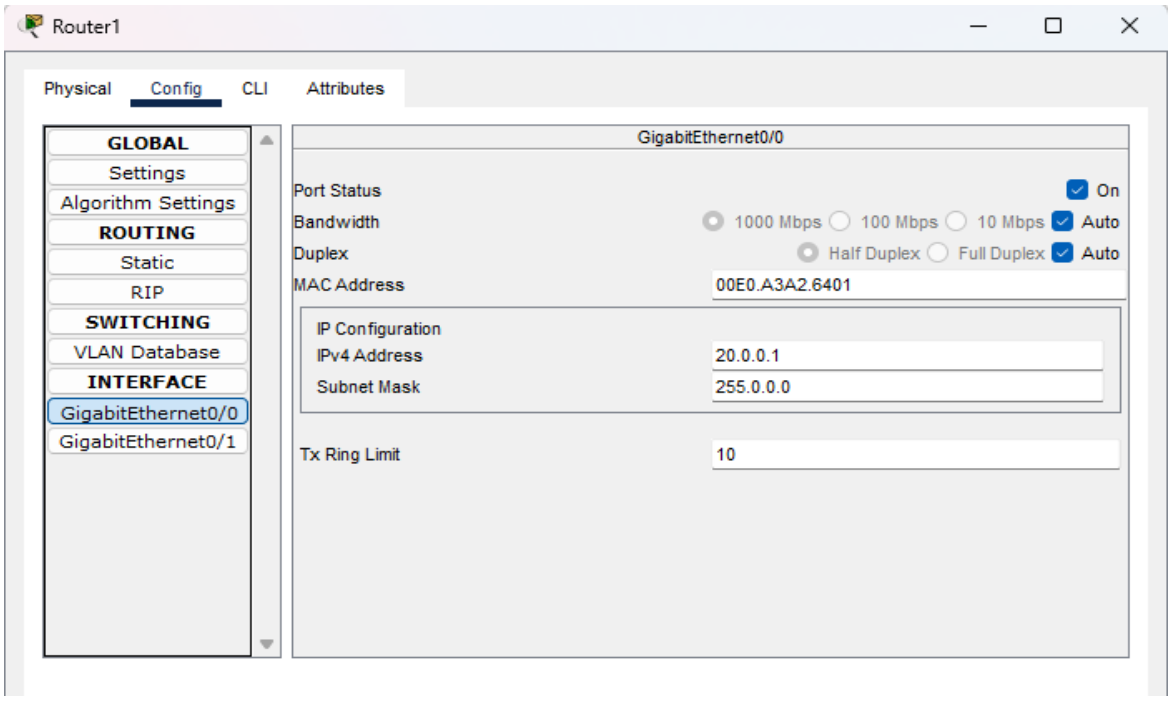

## Configure Router0

### Interface Gigabit Ethernet 0/1:

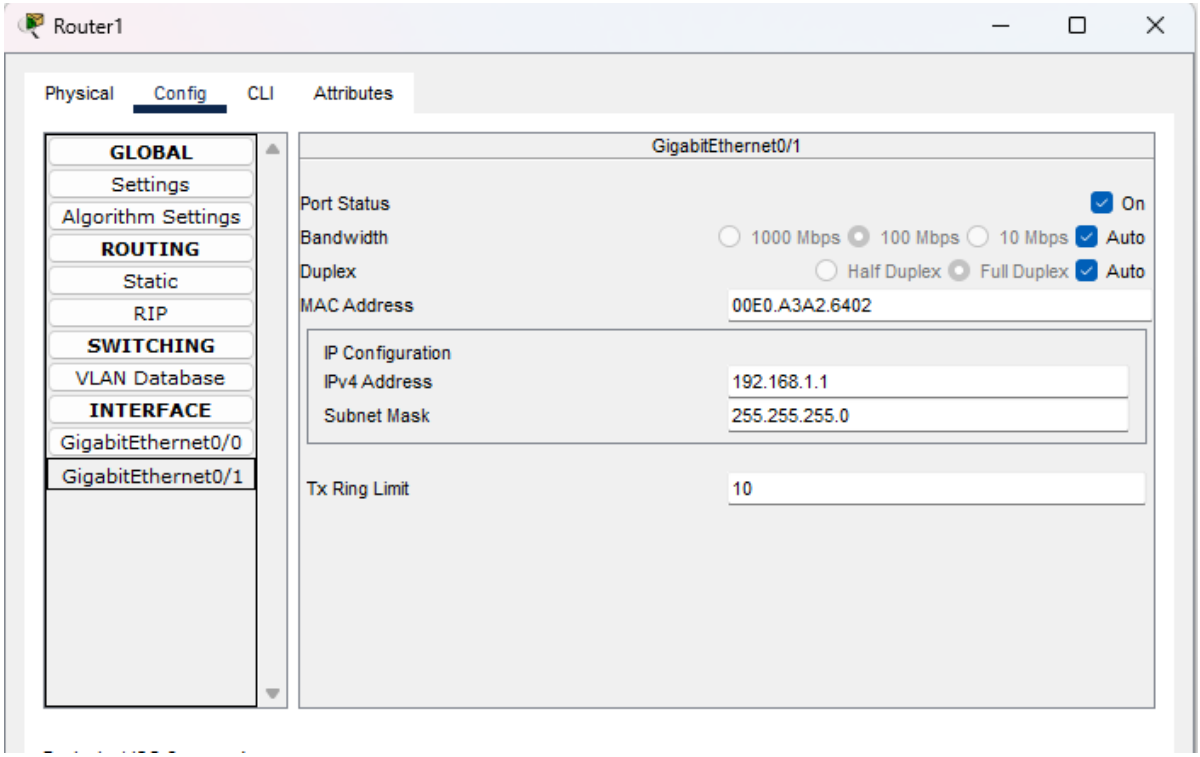


## Configuring Router1:

### Interface GigabitEthernet0/0:

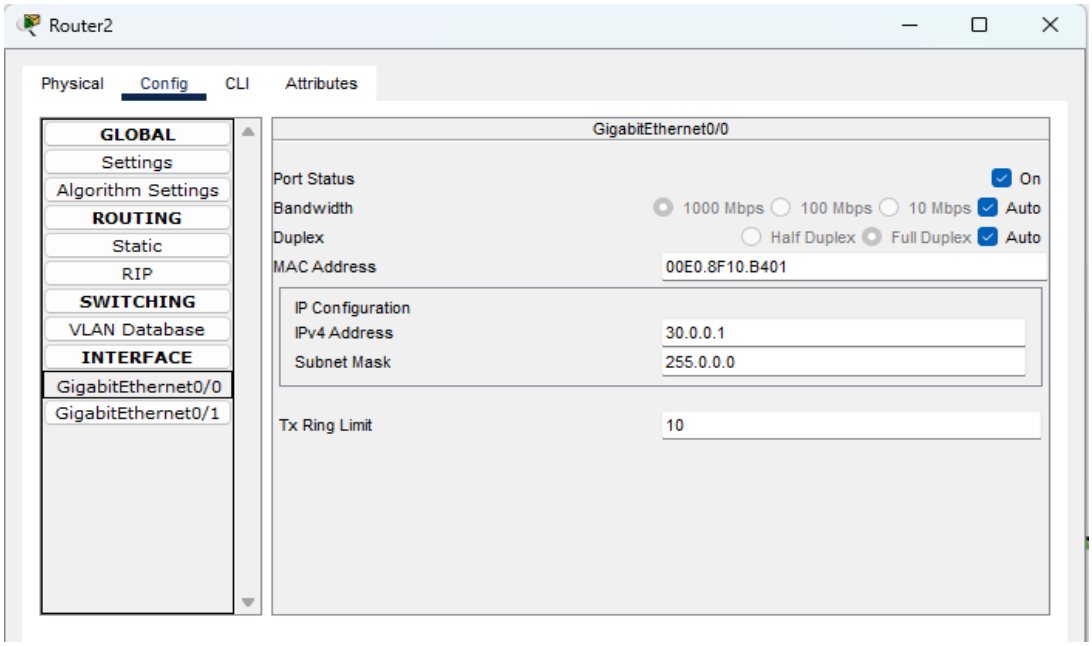



### Interface GiganitEthernet0/1:

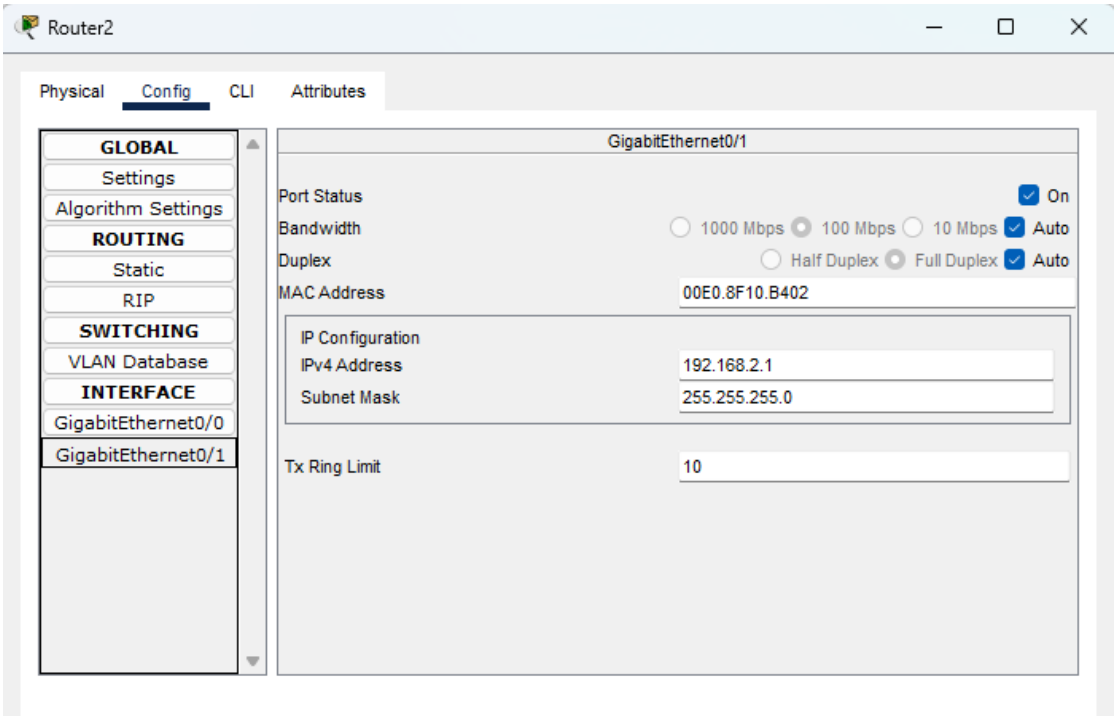


## Configuring Router2:

### Interface GigabitEthernet0/0:

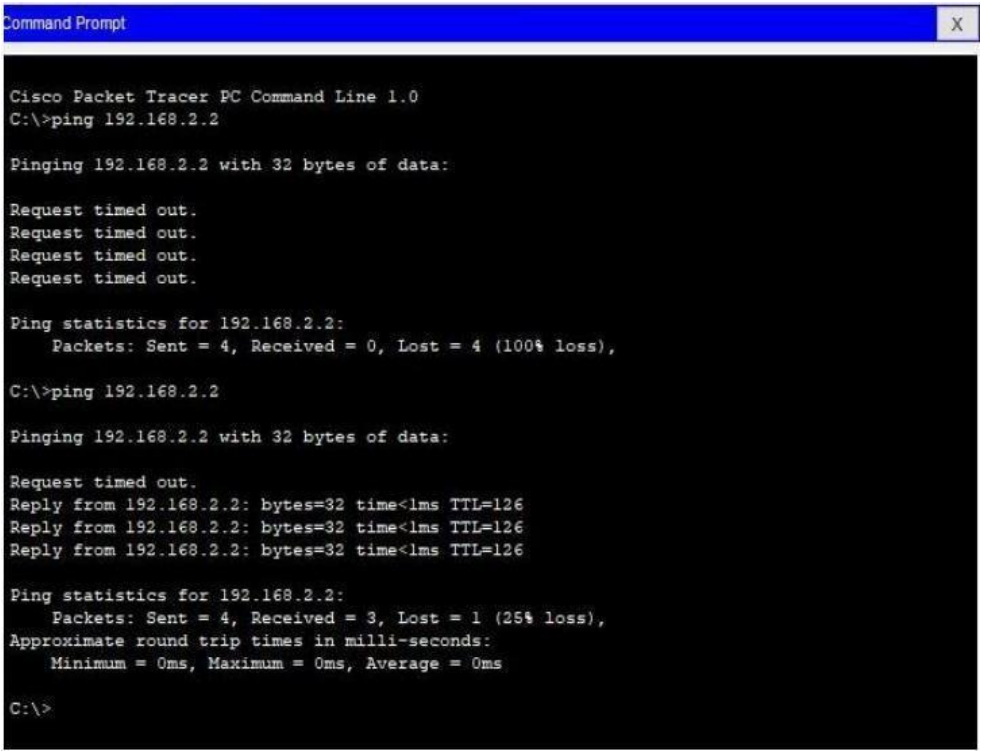



### Interface GigabitEthernet0/1:

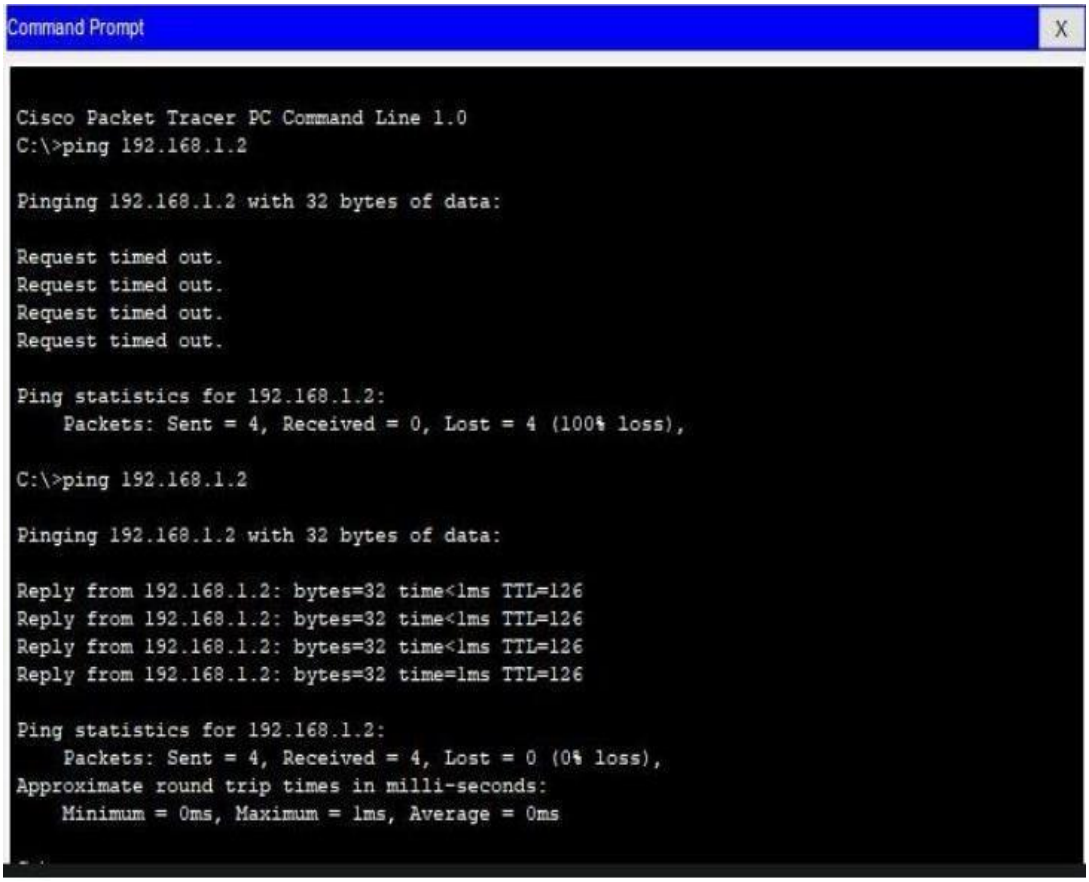


**Output:**

**Pinging PC2(192.168.2.2)**




**Pinging PC1(192.168.1.2**



# Practical No. 7

**Aim:** Web Security with SSL/TLS: Configure and implement secure web communication using SSL/TLS protocols, including certificate management and secure session establishment.

**Theory:**

**Web and Internet Security:**

### 1) Security on Internet:

Initially the Internet was intended to be open, public, and accessible by ordinary users.

Factors which were considered initially while describing the internet:

1. Decentralization
2. Redundancy
3. Asynchronous
4. Concurrency

### 2) Traffic Security approaches:

Approaches providing Web Security are as follows:

1. Network Level
2. Transport Level
3. Application Level

**Secure Socket Layer(SSL):**

Secure Socket Layer (SSL) is a Cryptographic Protocol. SSL

provides security over the internet.

Netscape developed SSL.

SSL is used for security management on the internet while message transmitting
SSL uses a program layer situated between HTTP and TCP layers. SSL is not

interoperable.

**Input:**

```
import ssl import socket

# Create a default SSL context
context = ssl.create_default_context()

# Create a socket
sock = socket.socket(socket.AF_INET)

# Wrap the socket with the SSL context
ssl_sock = context.wrap_socket(sock, server_hostname='example.com')

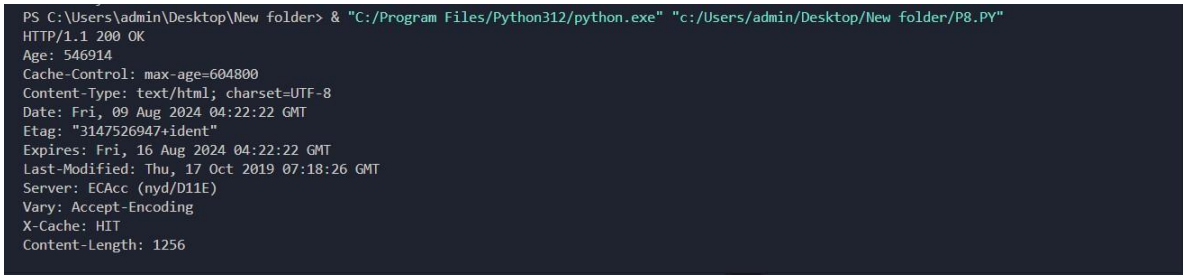
# Connect to the server
ssl_sock.connect(('example.com', 443))

# Send a request
ssl_sock.sendall(b'GET / HTTP/1.1\r\nHost: example.com\r\n\r\n')

# Receive the resopnse response = ssl_sock.recv(1024) print(response.decode())

# Close the socket
ssl_sock.close()
```

**Output:**



# Practical No. 8

**Aim:** Intrusion Detection System: Setup and configure an intrusion detection(IDS) to monitor network traffic and detect potential security breaches or malicious activities.
**Theory:**

## 1. Intrusion:

Intrusion is a process of attempting to break into system without permisssion or by force or misusing someones's system.

It is a set of events that comprises confidentiality, integrity and availability of resources.

It can be remote machine, system or physical.

Intrusion are the actions that breach the system's security policy.

## 2. Intrusion Detection:

Intrusion Detection is the knowledge and art of detecting when a network or system is improperly used or without authorization.

Monitoring of network and system activities and resources and using its information to find when an intrusion is possible is known as Intrusion Detection System (IDS).

Intrusion Detection is basically the process used to discover intrusions.

**Input:**

```
import cv2

import numpy as np

# Load the image

img = cv2.imread('download.jpeg')

# Convert to grayscale

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply thresholding to highlight potential intrusions

thresh = cv2.threshold(gray, 170, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)[1]

# Define a kernel for morphological operations

kernel = np.ones((10, 10), np.uint8)

# Erode to remove small noise points, then dilate to close gaps in contours

eroded = cv2.erode(thresh, kernel, iterations=2) dilated = cv2.dilate(eroded, kernel, iterations=2)

# Find contours in the dilated image

contours, _ = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Flag to check if any intrusion is detected
```

```
intrusion_detected = False

# Process each contour

for contour in contours:

x, y, w, h = cv2.boundingRect(contour) aspect_ratio = float(w) / h

area = cv2.contourArea(contour)

# Define criteria for an intrusion

if aspect_ratio > 1.5 and area > 500:

# Draw a green bounding box around detected intrusion cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2) print(f"Intrusion detected at location: ({x}, {y}) with width: {w} and

height: {h}")

intrusion_detected = True

# Display the result

cv2.namedWindow("Intrusion Detected!!", cv2.WINDOW_NORMAL)
height, width = img.shape[:2] cv2.resizeWindow("Intrusion Detected!!", width, height)
cv2.imshow("Intrusion Detected!!", img)

# Save the output image
cv2.imwrite("output.jpg", img)

# Inform the user if no intrusions were detected
if intrusion_detected: print("Intrusion Detected")
else:
print("No intrusion detected.")

# Wait for a key press and close all windows
cv2.waitKey(0) cv2.destroyAllWindaows()a
```

**Output:**



# Practical No. 9

**Aim:** Malware Analysis and Detection: Analyze and Identity malware samples using antivirus tools, analyze their behavior and develop counter measures to mitigate their impact.

**Theory:**

**Malware:**

Malware means malicious software. These software helps hackers to gather sensitive information, interrupt user's computer's operations, unauthorized access to computer.It can be in a form of annoying script or code or software.

Some common types of malware include:

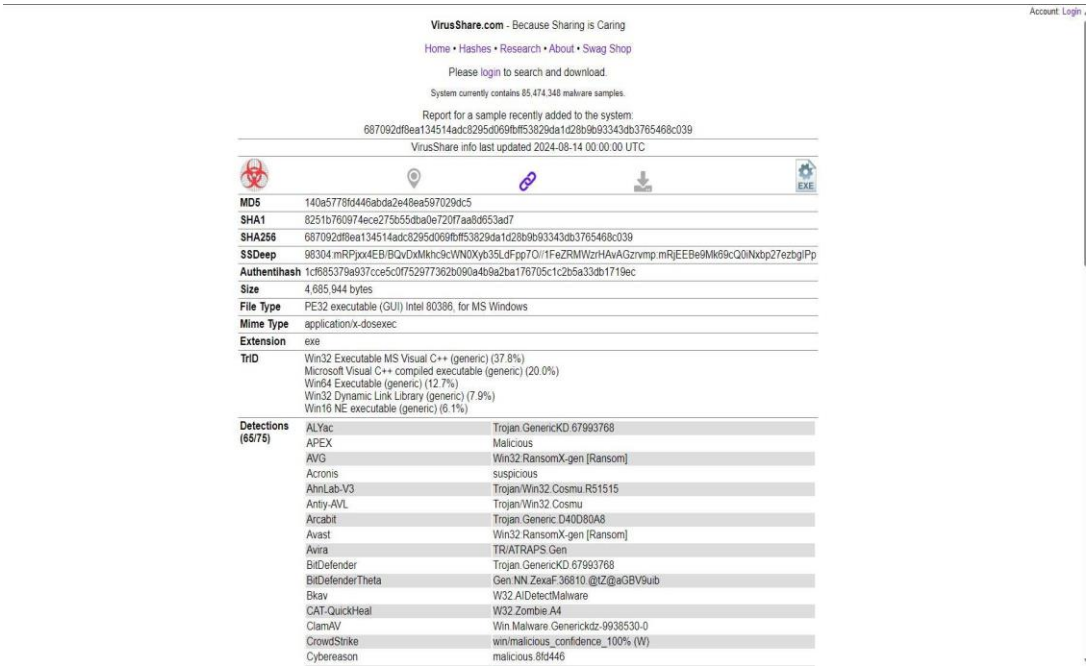
1. Viruses: Viruses are malicious programs that attach themselves to legitimate files or programs and spread by infecting other files. When infected files are executed, the virus replicates and spreads further, potentially causing damage to data and systems.

2. Worms: Worms are standalone programs that replicate and spread across computer networks without needing to attach themselves to other files. They often exploit security vulnerabilities to self-propagate and can cause network congestion and data loss.

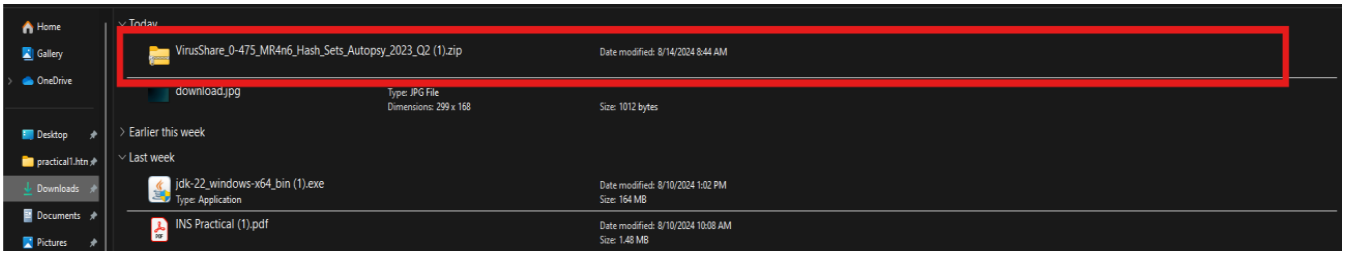
3. Trojans: Trojans are deceptive programs that disguise themselves as legitimate software, tricking users into installing them. Once installed, Trojans can perform a variety of malicious activities, such as stealing sensitive information, opening backdoors, or launching attacks.

4. Ransomware: Ransomware encrypts a user's files or entire system and demands a ransom payment in exchange for providing the decryption key. It can lead to data loss and significant disruption if not properly managed

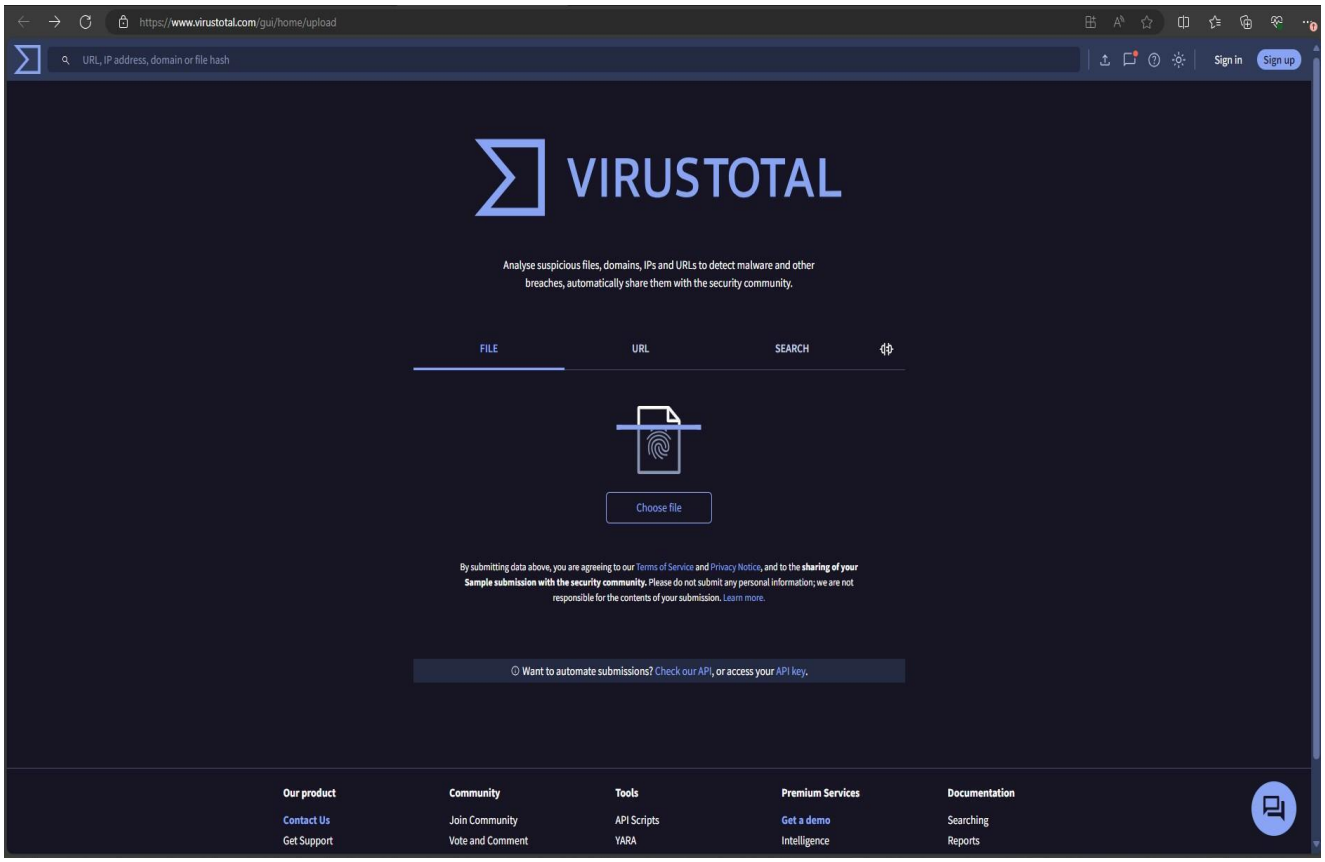
**Step 1:** Go to the website www.virusshare.com for downloading the clean sample of Malware



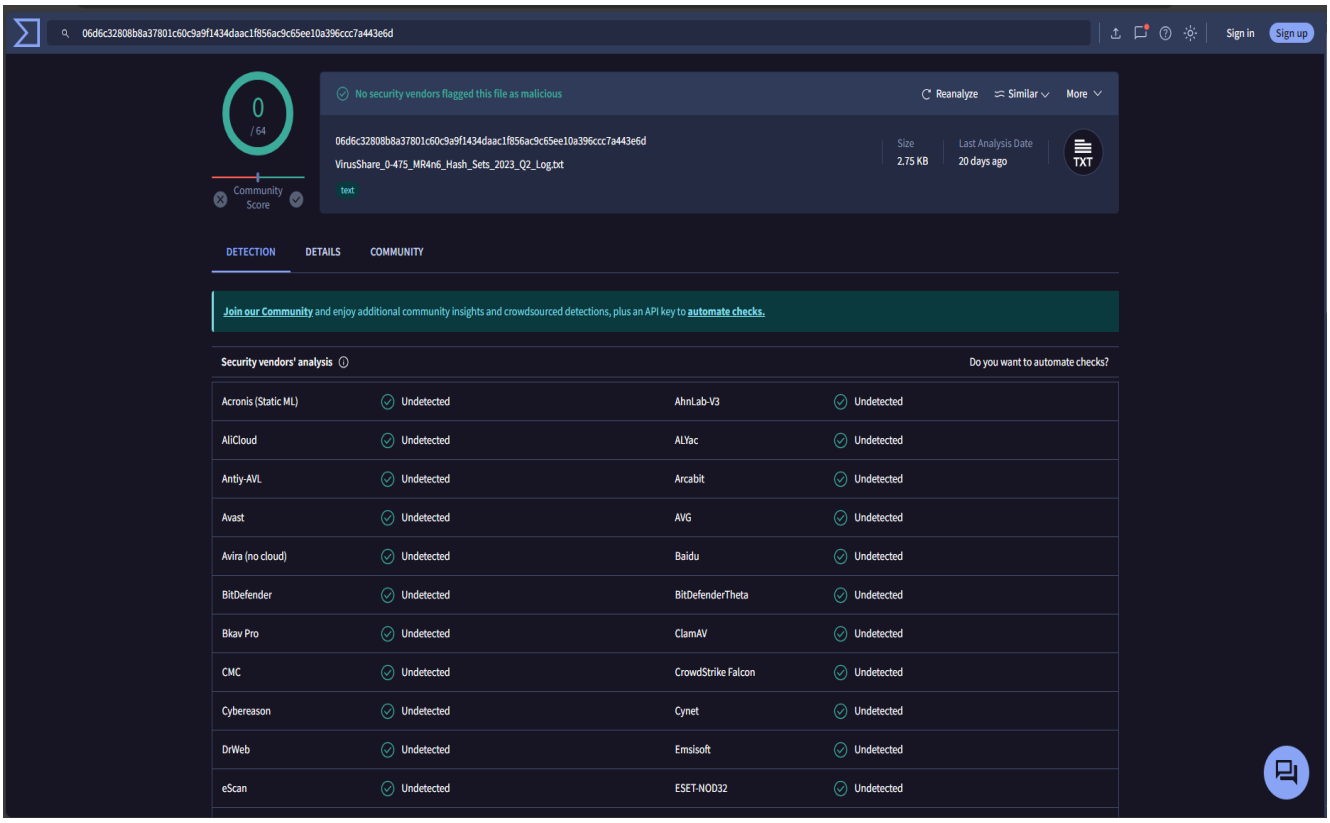
**Step 2:** By clicking the above download icon the Malware gets downloaded in ZIP format.

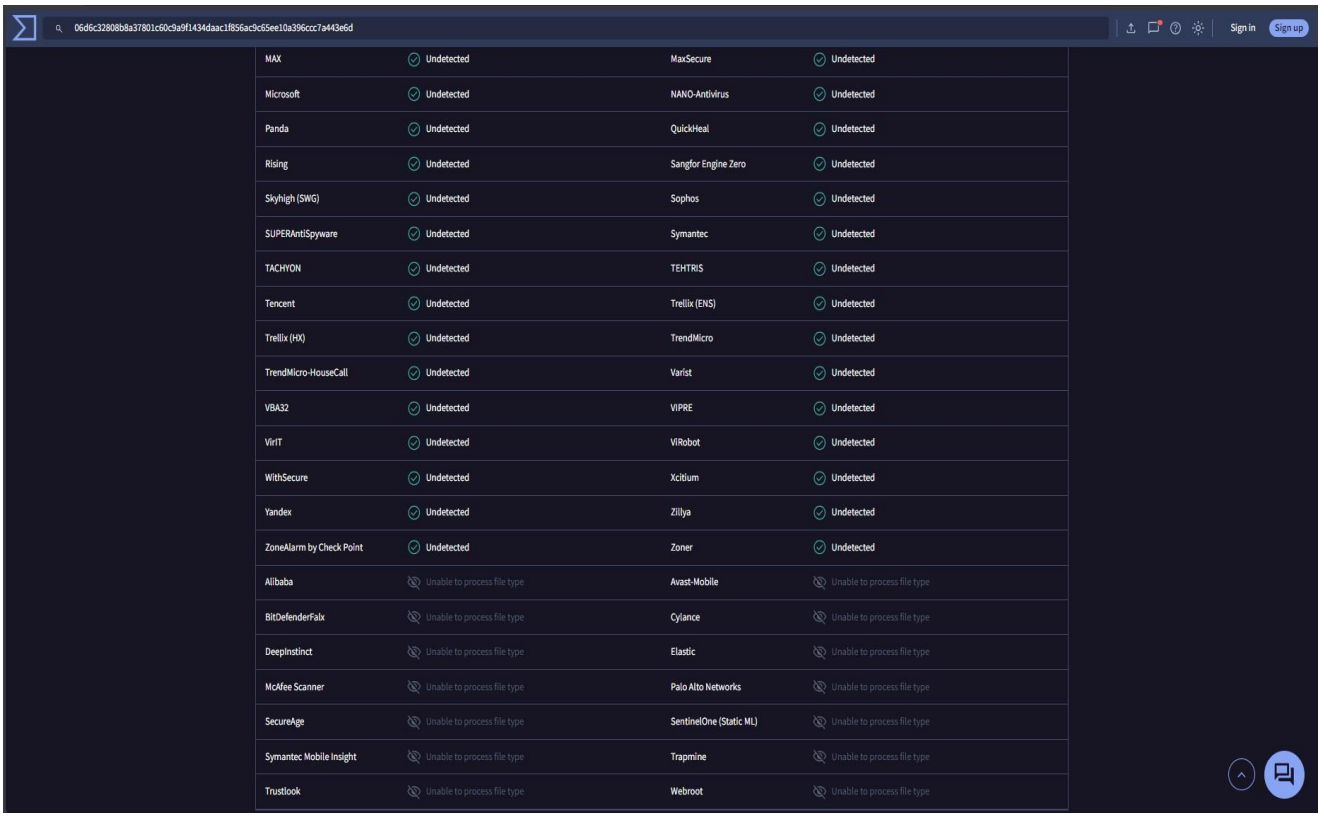



**Step 3:** In order to analyse the Malware, we select the website [www.virustotal.com](http://www.virustotal.com).



**Step 4:** Click on “Choose File” and select the file from the location.



# Practical No. 10

**Aim:** Firewall Configuration and Rule-based Filtering: Configure and test firewall rules to control network traffic, filter packets based on specified criteria, and protect network resources from unauthorized access.

**Theory:**

**Firewall:**

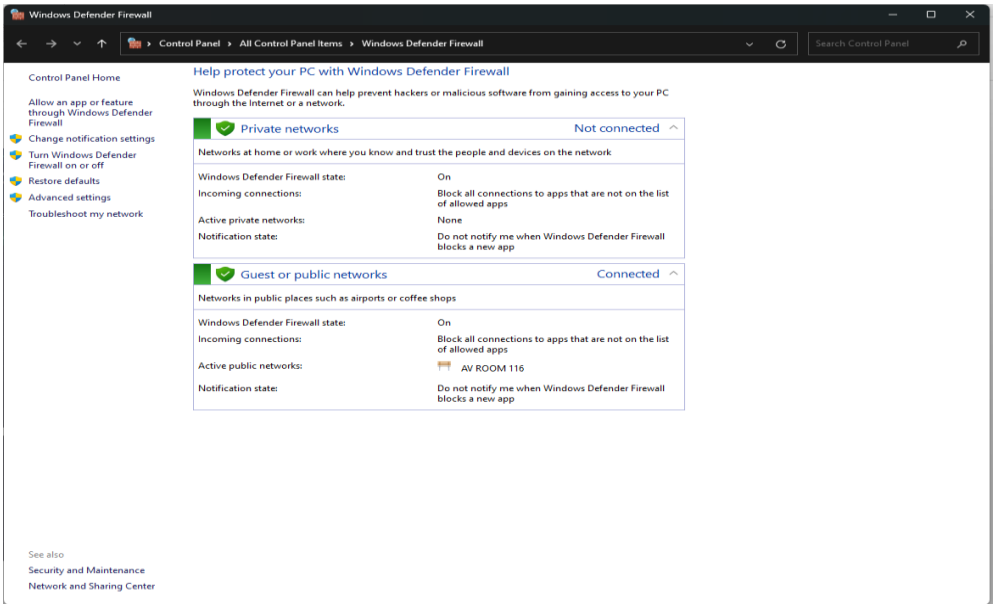
Firewall acts like a guard between your computer and internet. It checks if the traffic the traffic should continue on network or stop towards sending to destination. It keeps unwanted access out allows only designated traffic to enter or leave the computer.

Firewall sees each packet which enters into the network and leaves it. It also determines packets based on their origin and destination point and also contents on it. Firewall checks individual packets which are allowed to pass through it and checks whether the packets of similar patterns should be pass through it or not for future use. Firewall checks which programs can get connected to network.

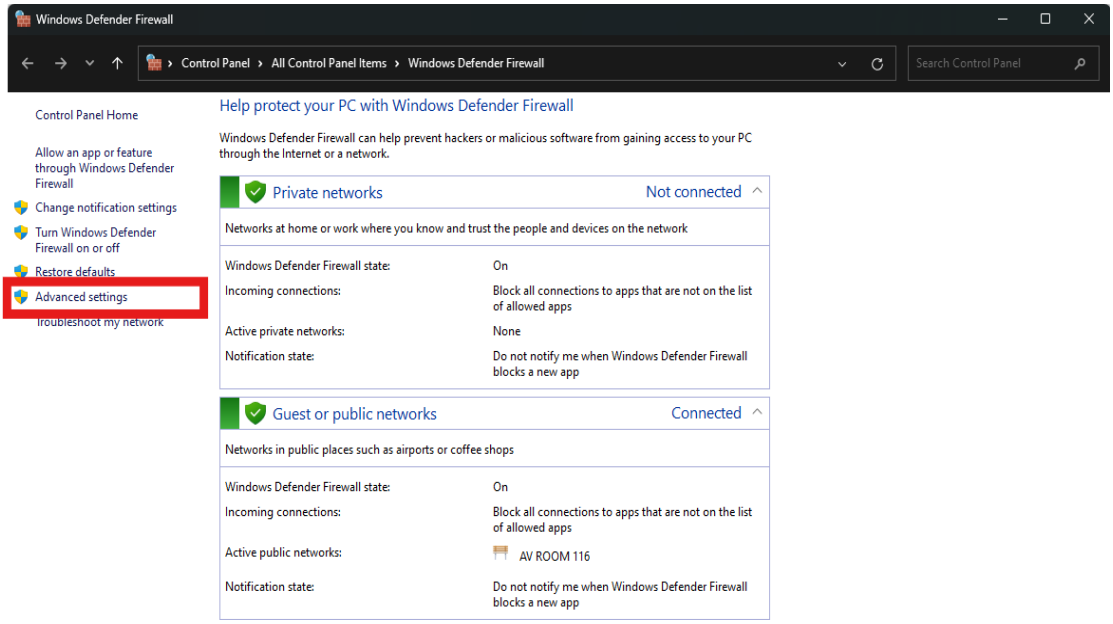
Types of Firewall:

1. Packet - filtering routers.
2. Application - level gateways
3. Circuit - level gateways
4. Stateful inspection firewall

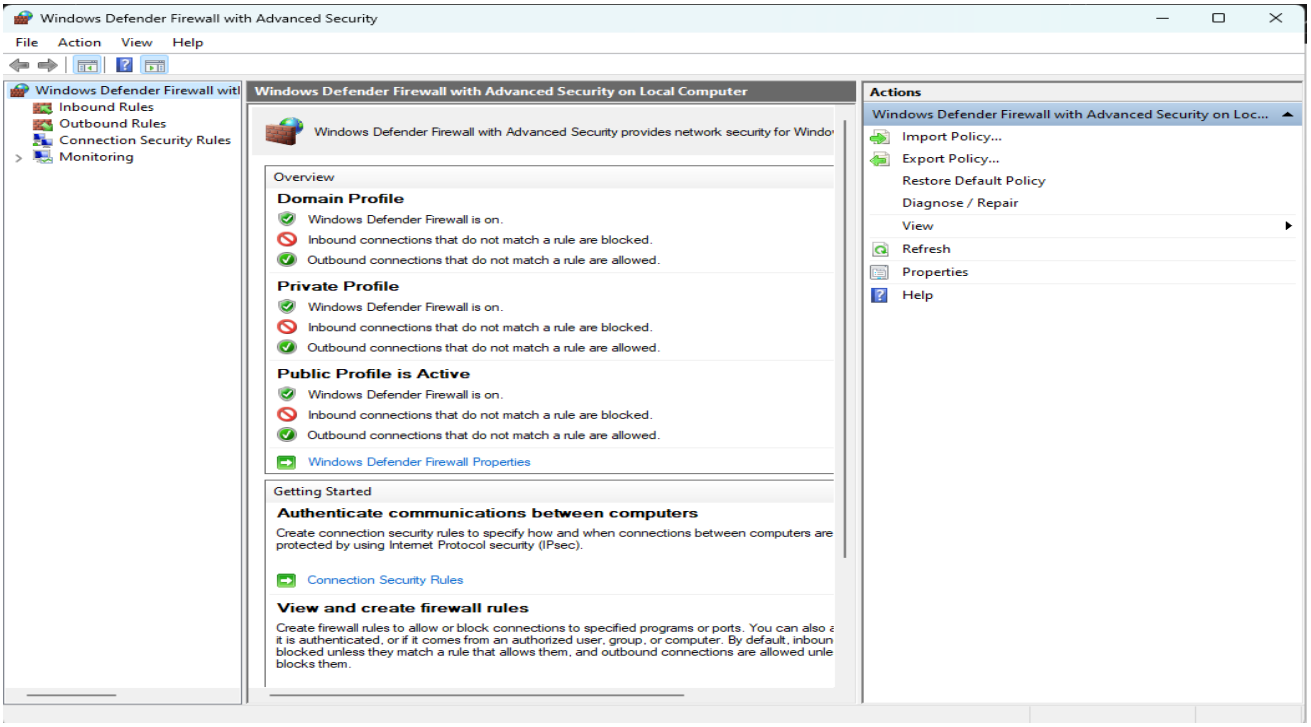
**Step 1:** Open 'Windows Defender Firewall'.

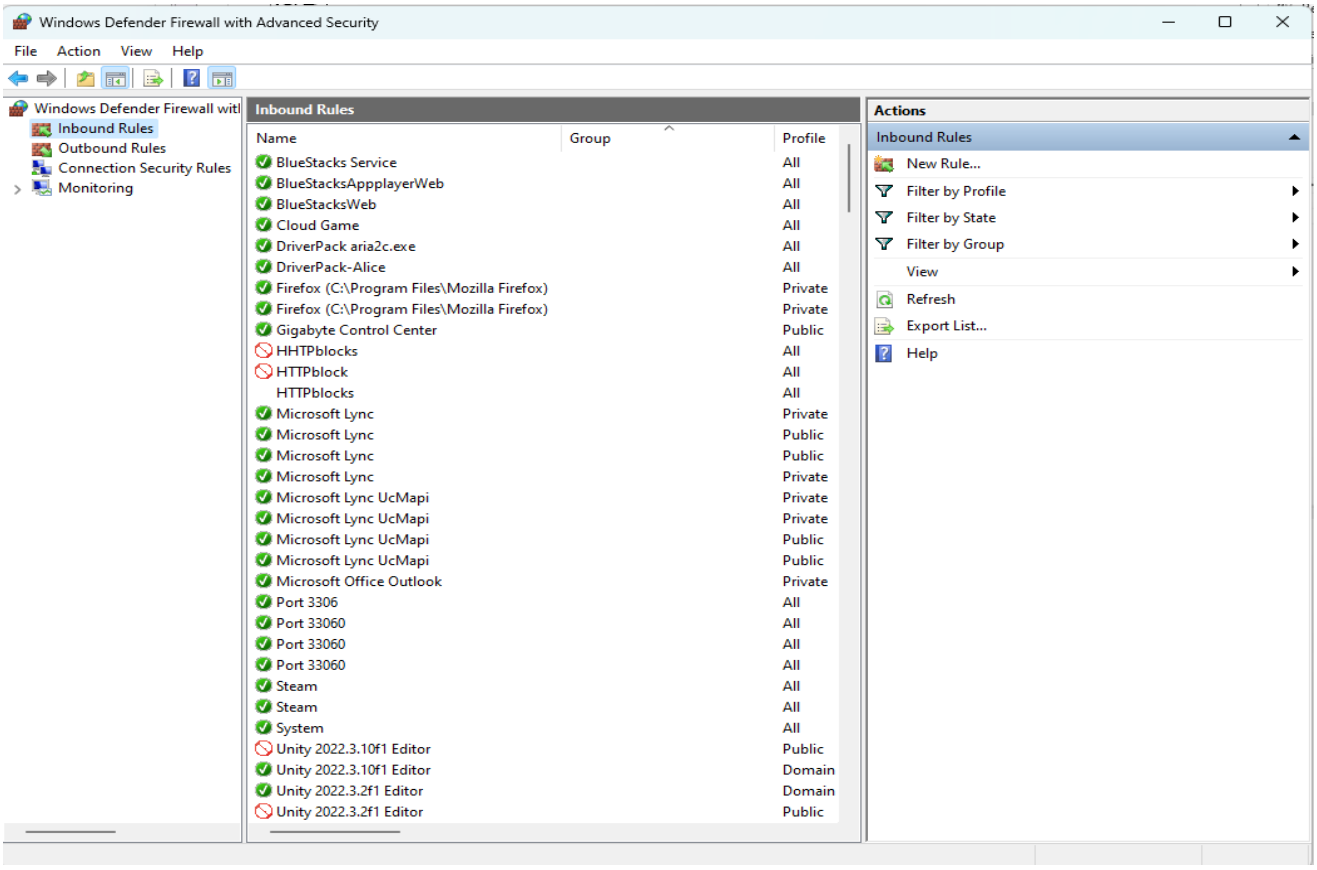


**Step 2:** Next we click on ‘Advanced settings.

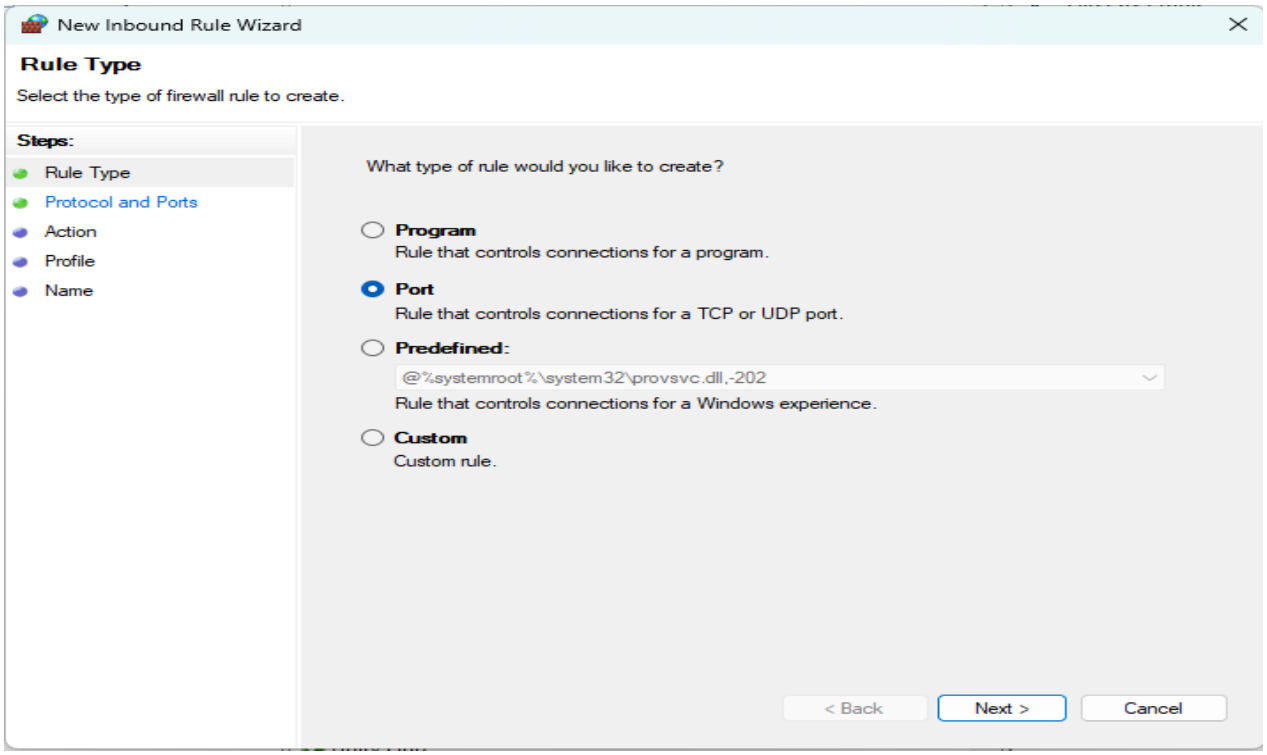



**Step 3:** Next we click on ‘Inbound Rules.

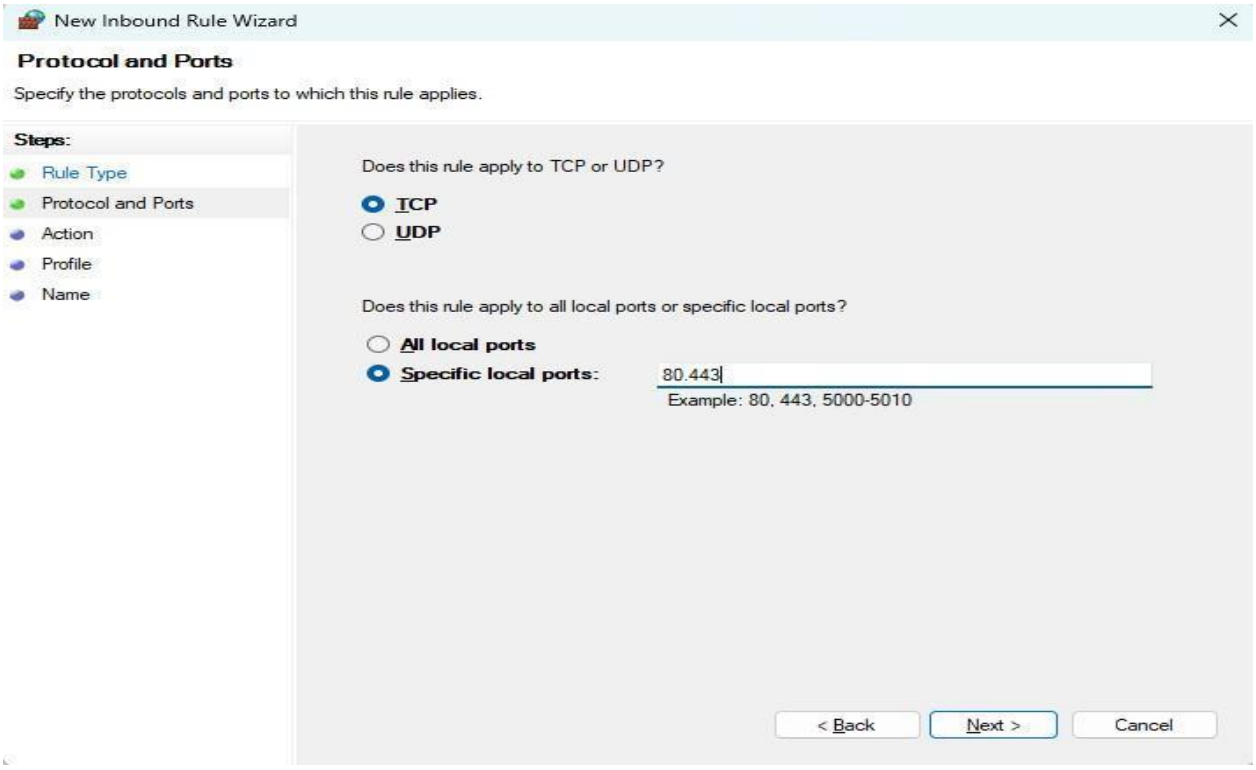




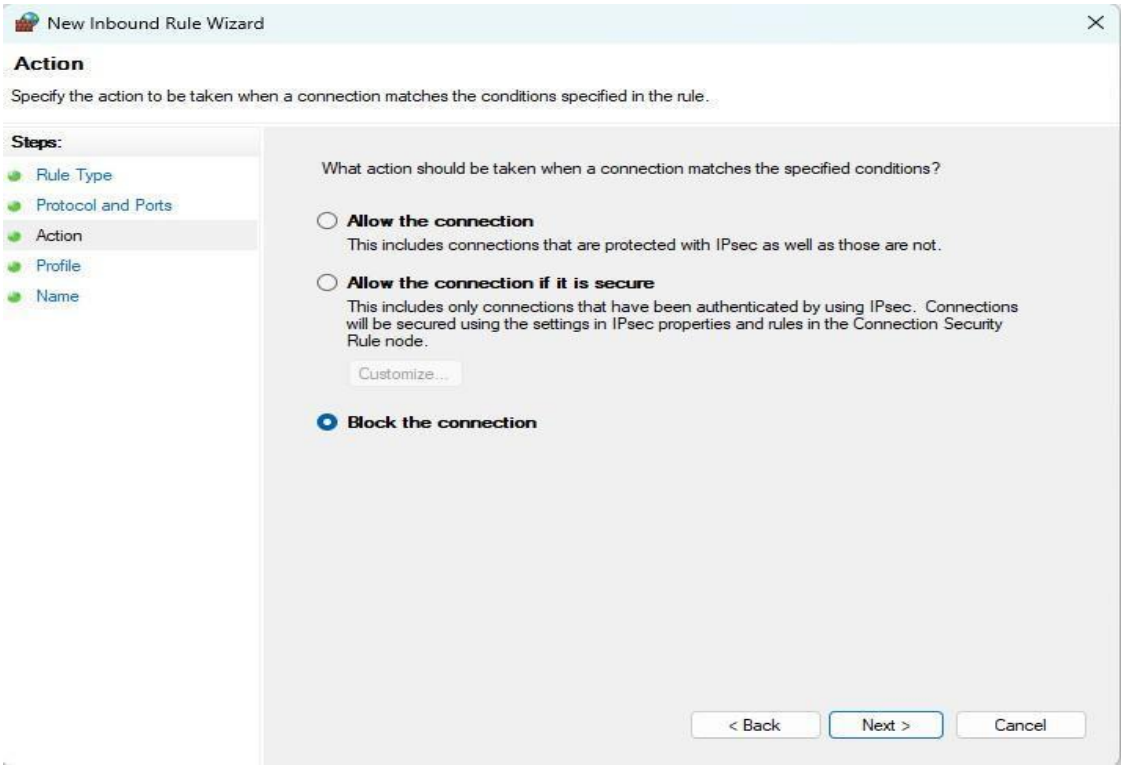

**Step 4:** Then click on ‘New Rule’



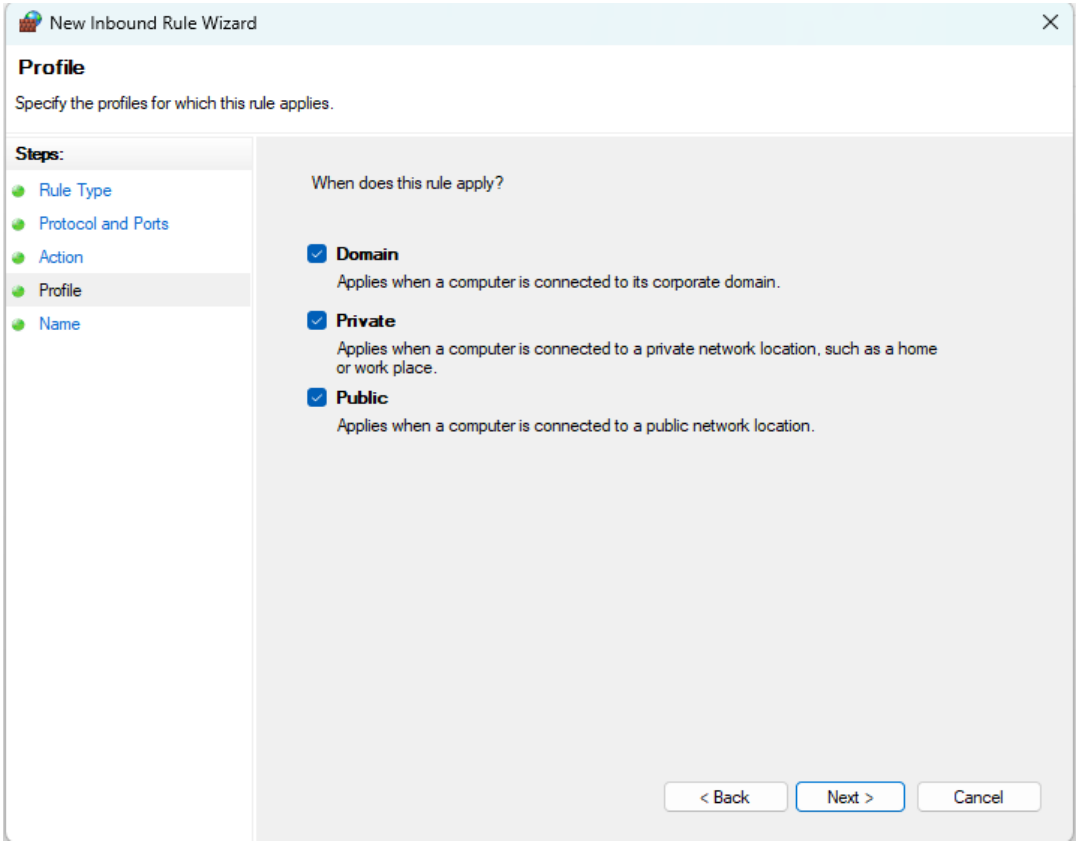
**Step 5:** Select the radio button ‘Port’ and click ‘Next’ and enter the following.



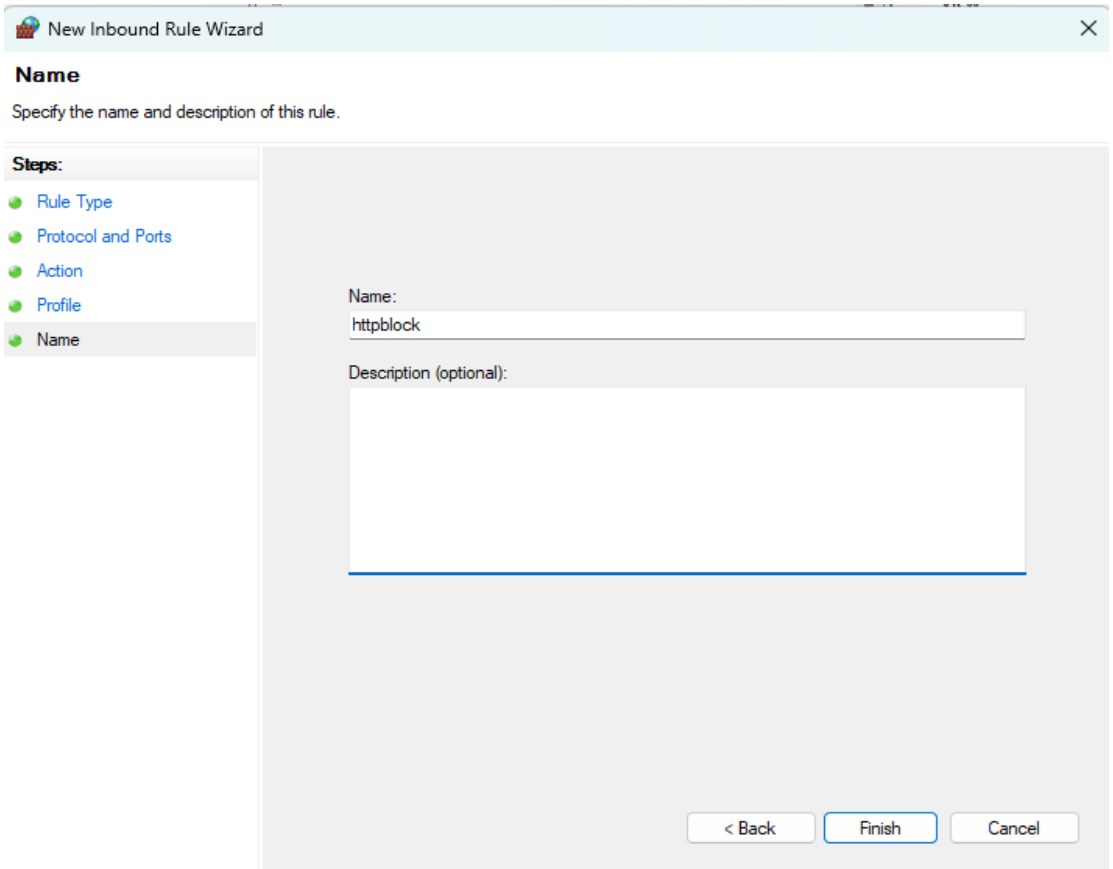

**Step 6:** After next, we need to finalise the rule



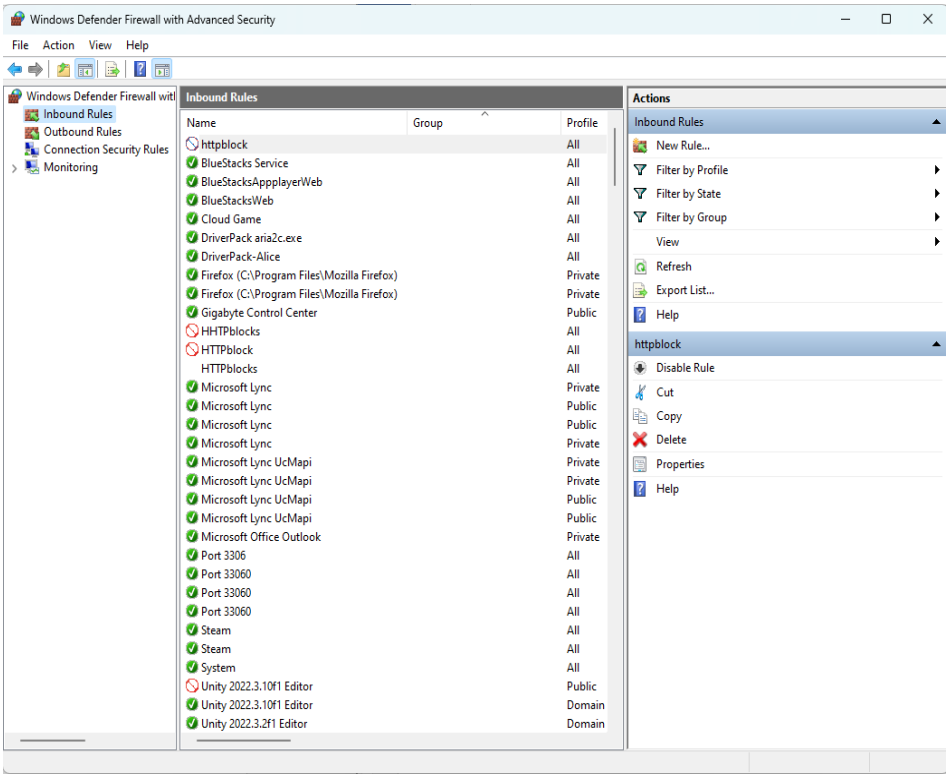
**Step 7:** Click ‘Next’ and we get the following




**Step 8:** After clicking the ‘Next’ button we need to name the rule and click finish



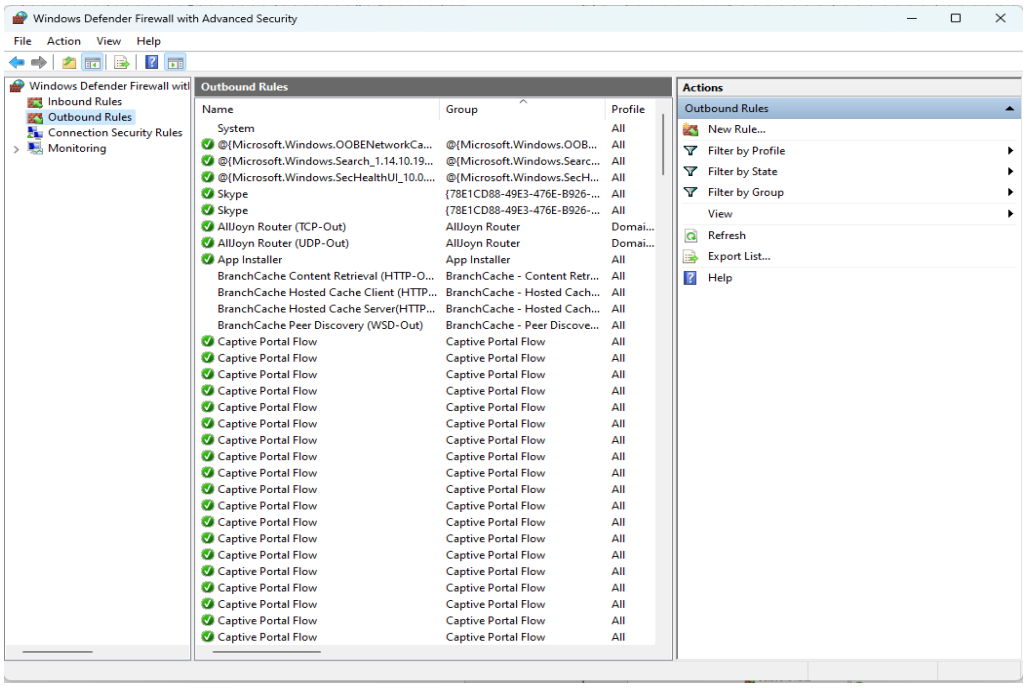
**Step 9:** The Inbound rule is added



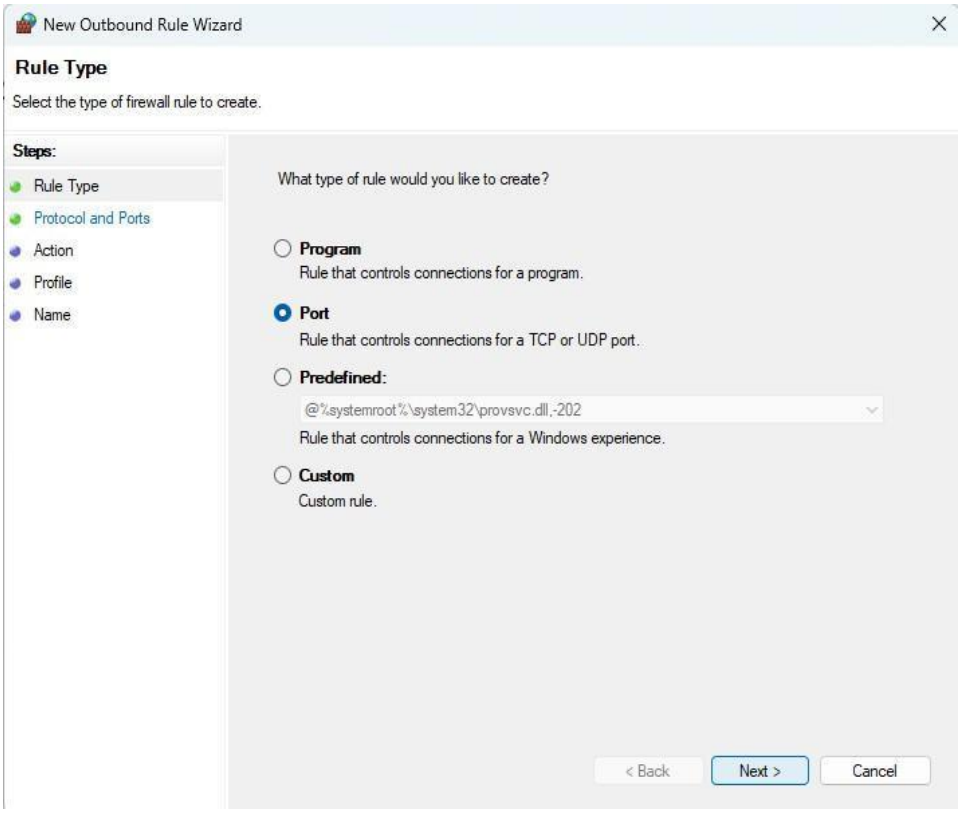

Same steps for Outbound rules

**Step 1: C**lick on ‘Outbound Rules.

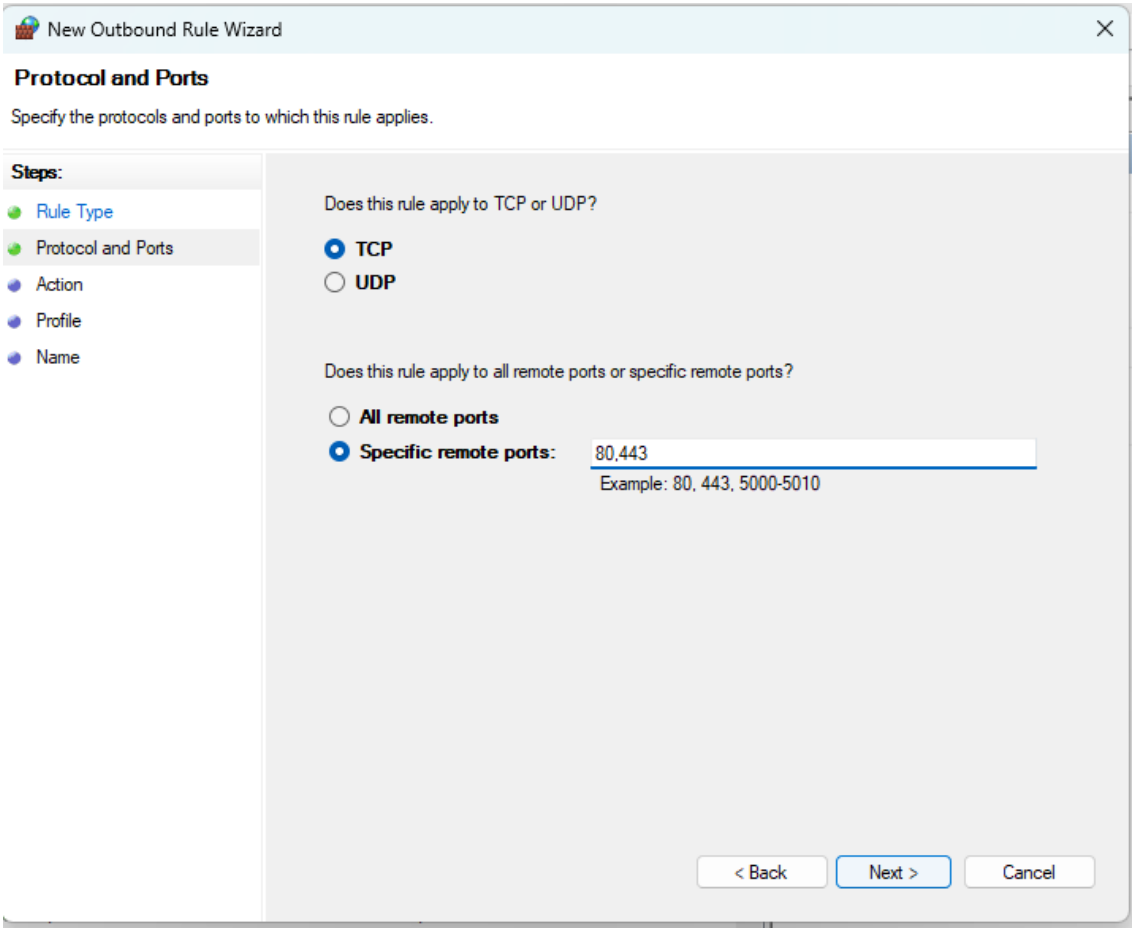
**Step 2:** Then click on ‘New Rule’ .



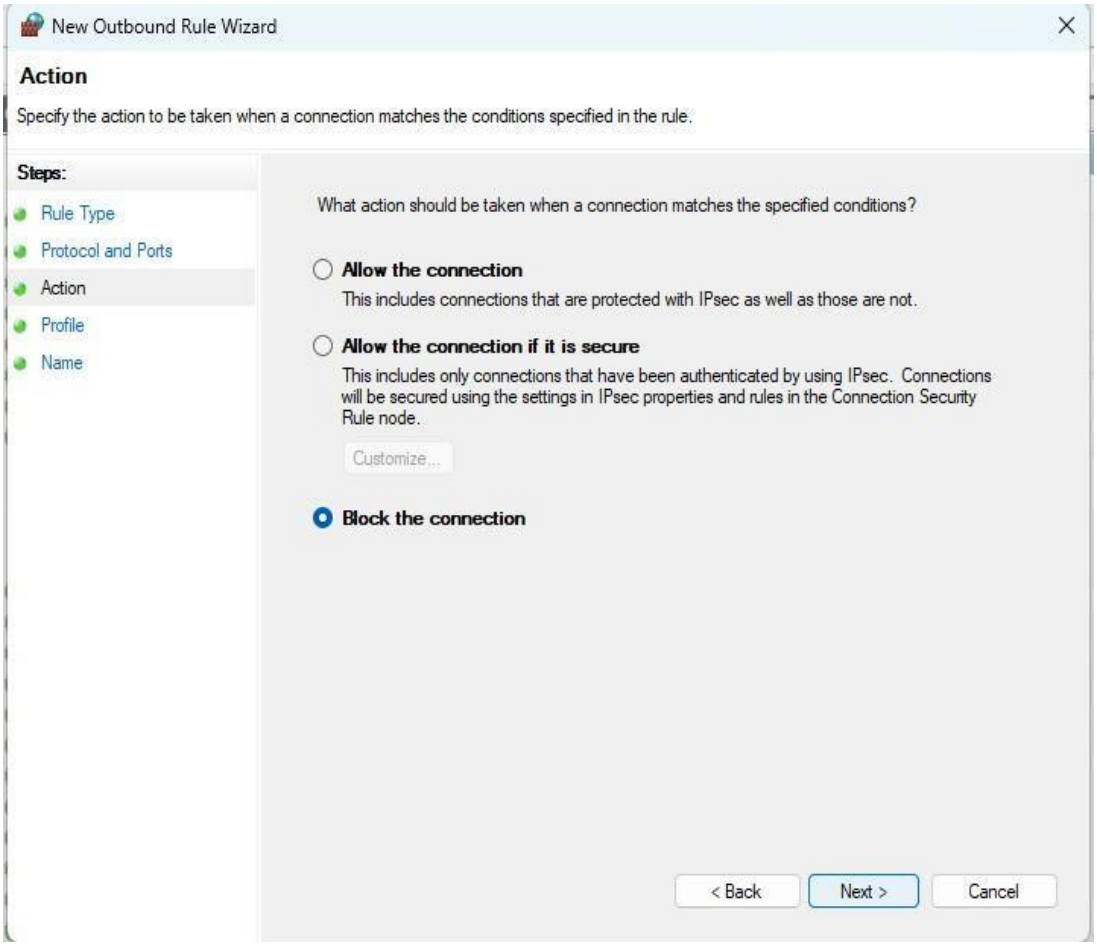
**Step 3:** Select the radio button ‘Port’ and click ‘Next’ and enter the following.



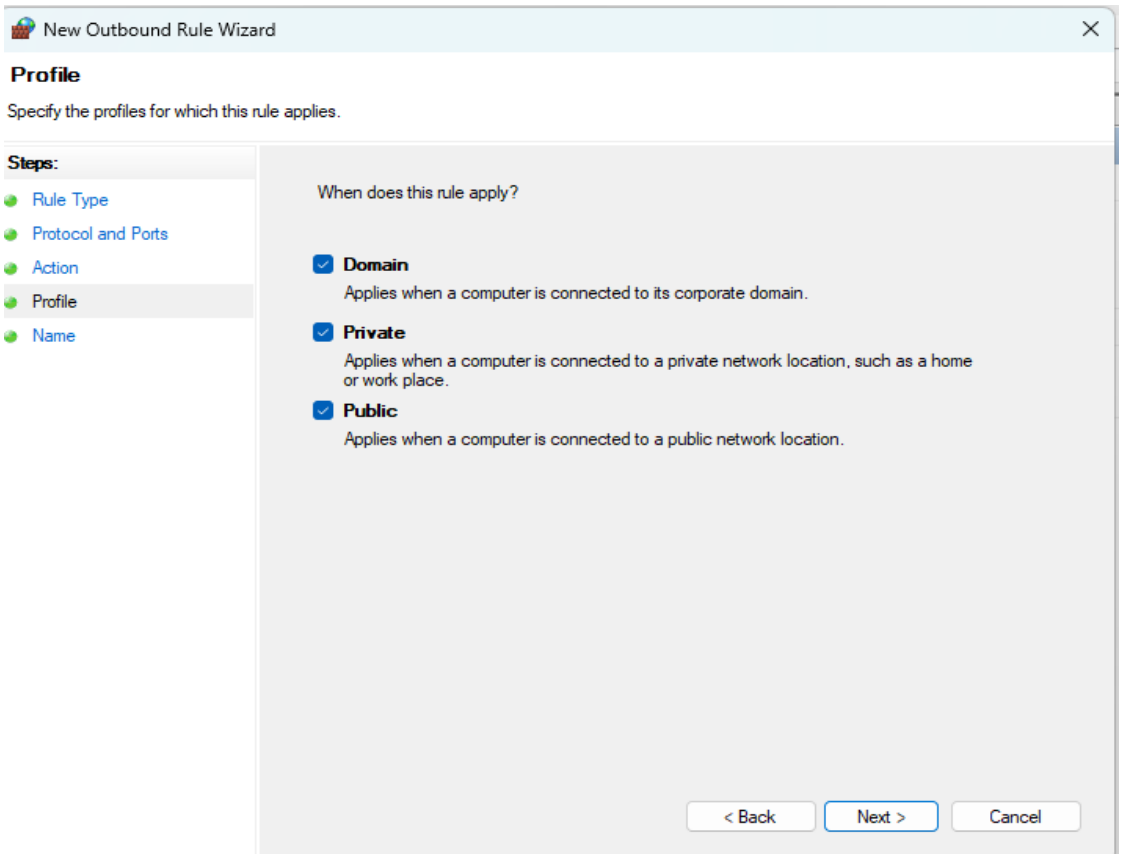

Steps 4 : click TCP and click on specific remote ports and enter 80,443.enter the Next .



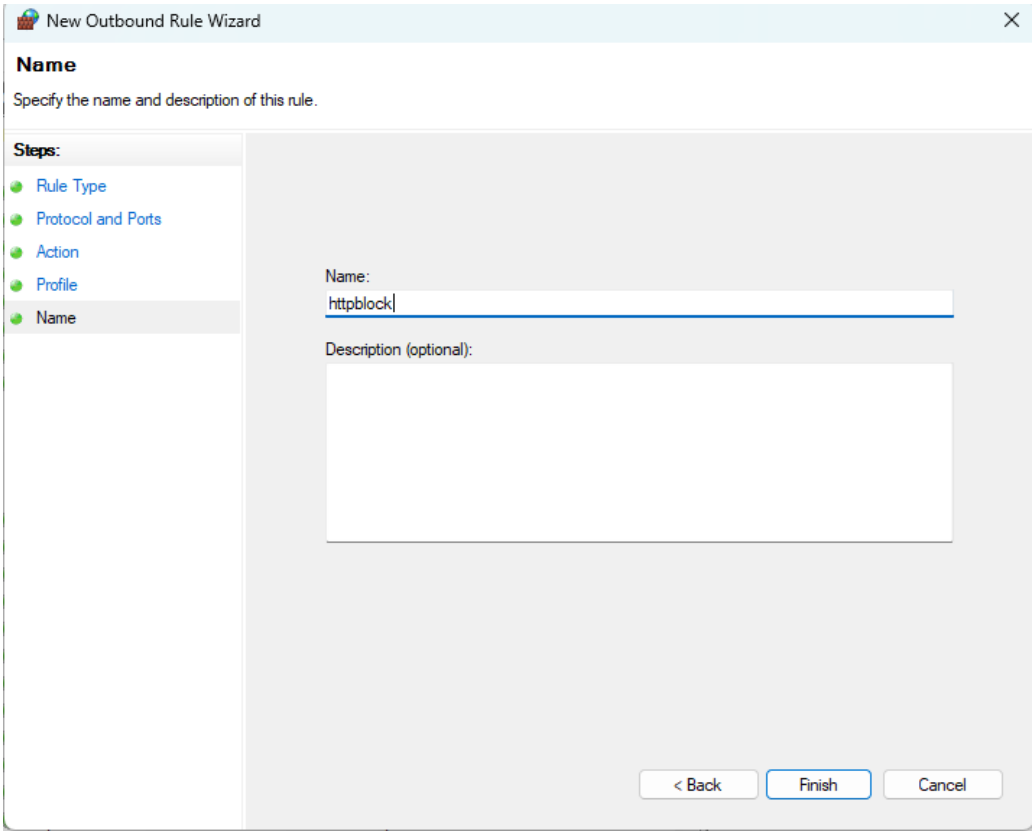
**Step 5:** After next, we need to finalise the rule



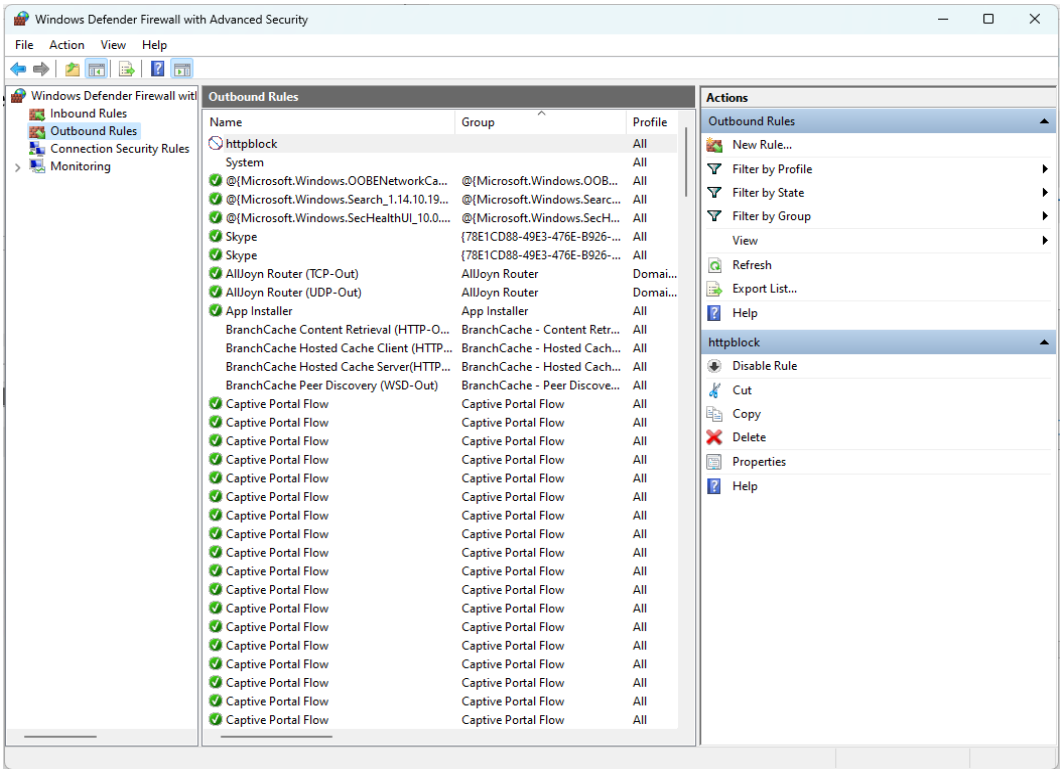

**Step 6:** Click 'Next' and we get the following



**Step 7 :** After clicking the ‘Next’ button we need to name the rule and click finish.



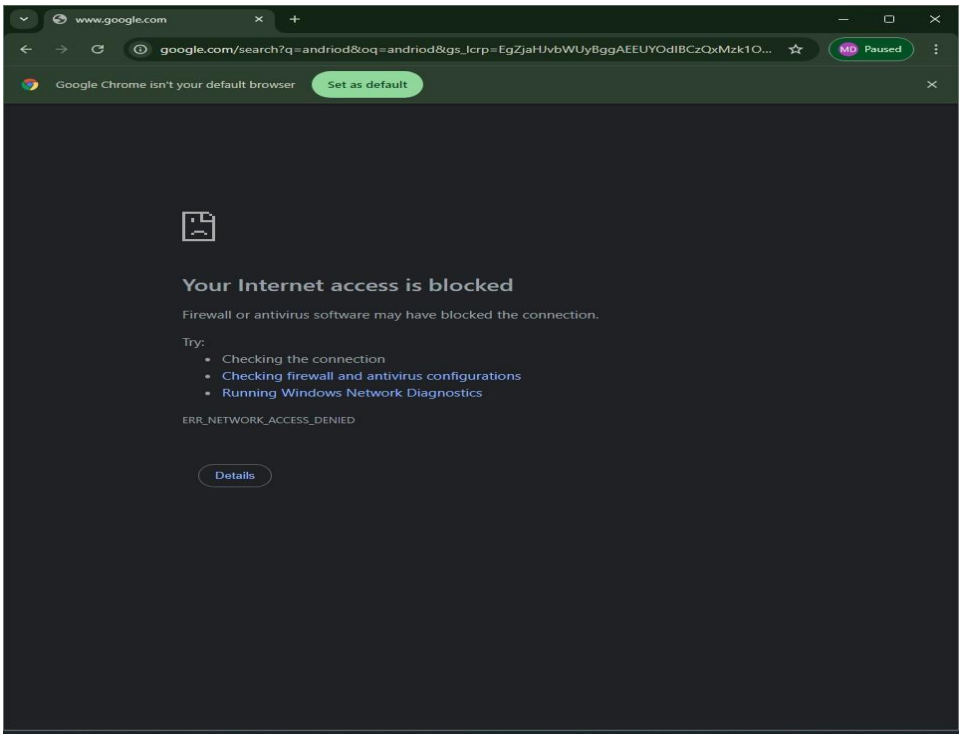

**Step 8:** The Outbound rule is added.



**Step 8:** The Outbound rule is added.

**Output:**

**A)** Try to access the internet. We see that the accessed is blocked.

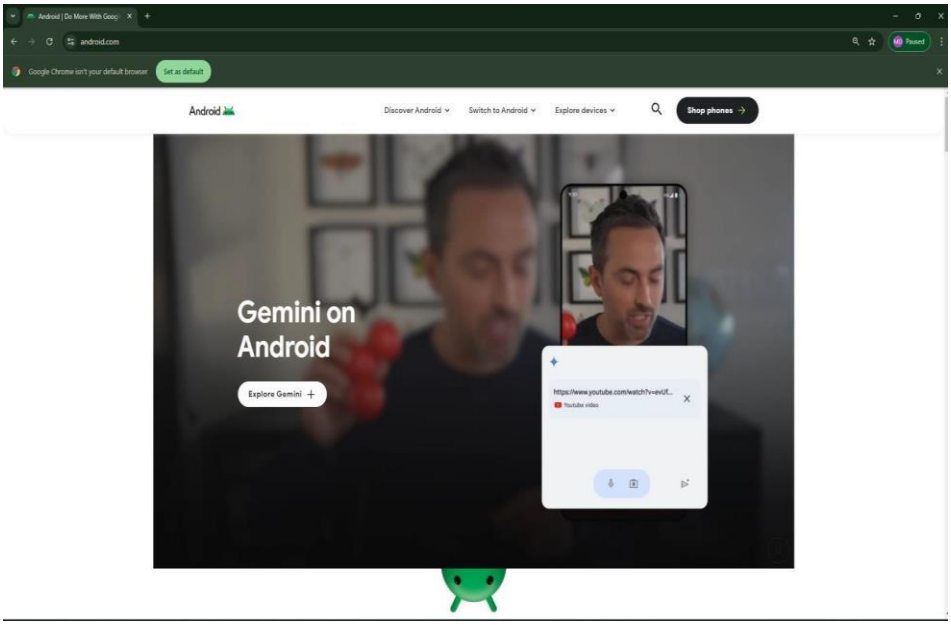



**B) Blocking Site**

**www.andriod.com**

**Input:**

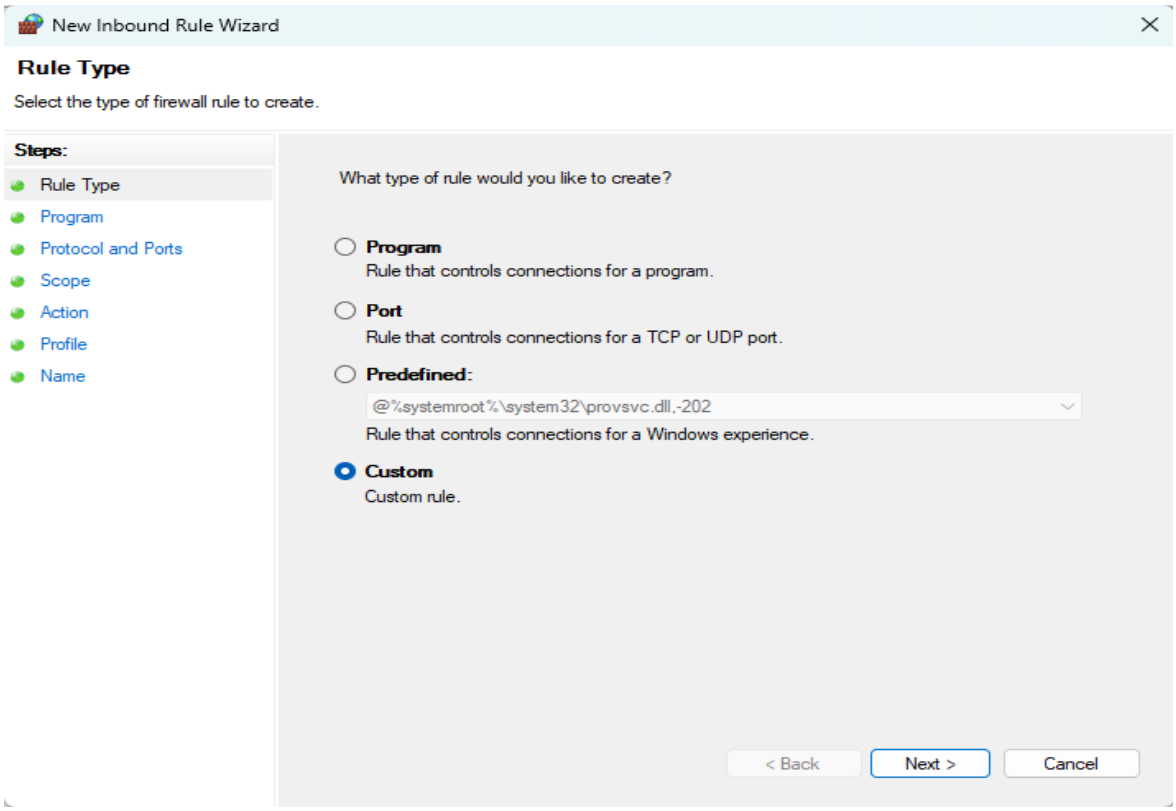
**Step 1:** We open the browser and access the website, which is now accessible.



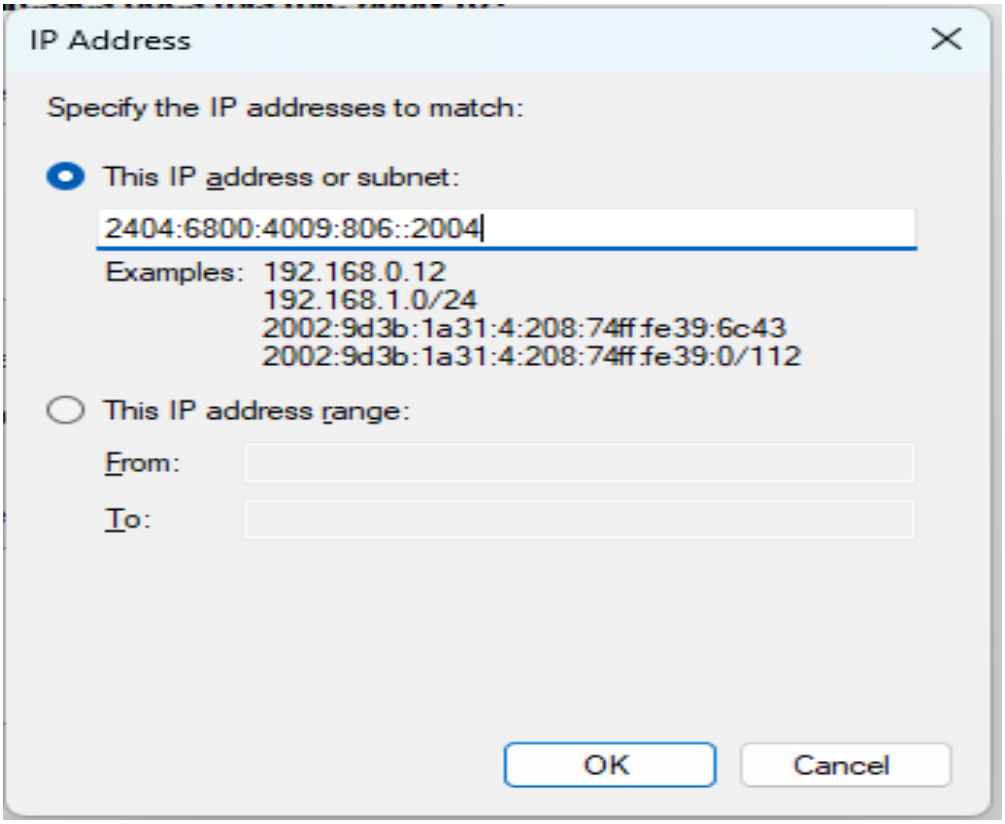
**Step 2:** We find the IP addresses of the website using the following command

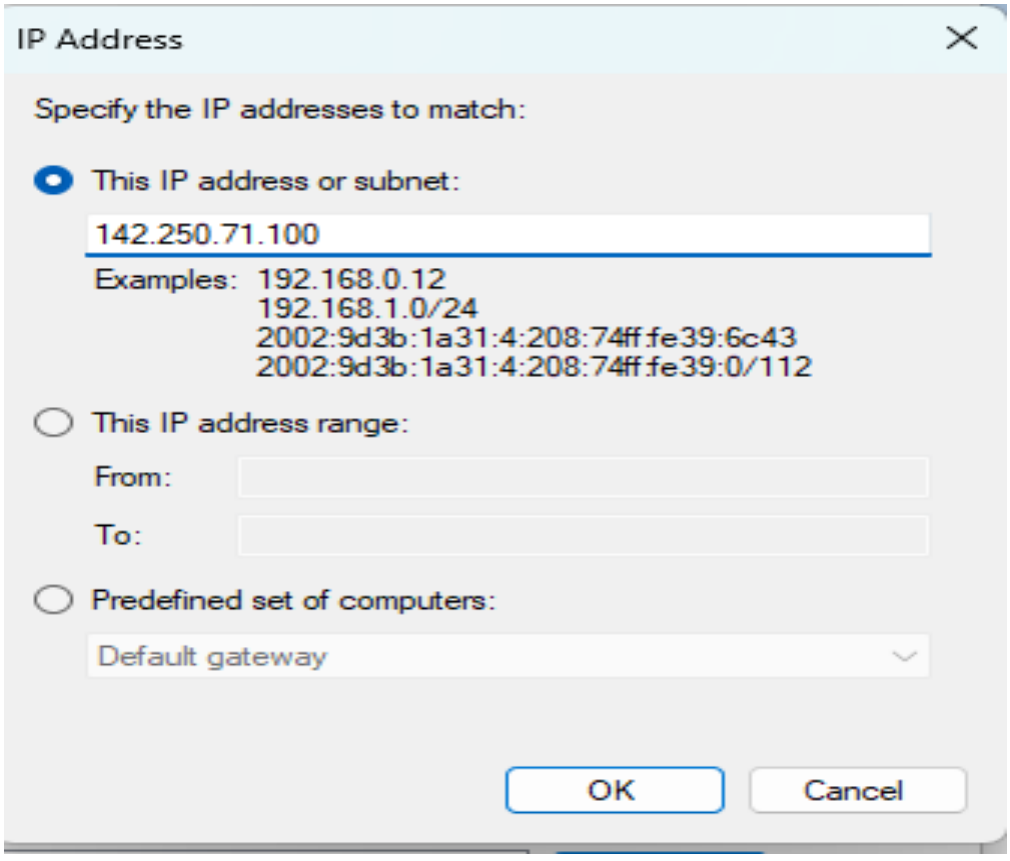



**Step 3:** We open the windows Firewall settings and apply the Inbound Rule

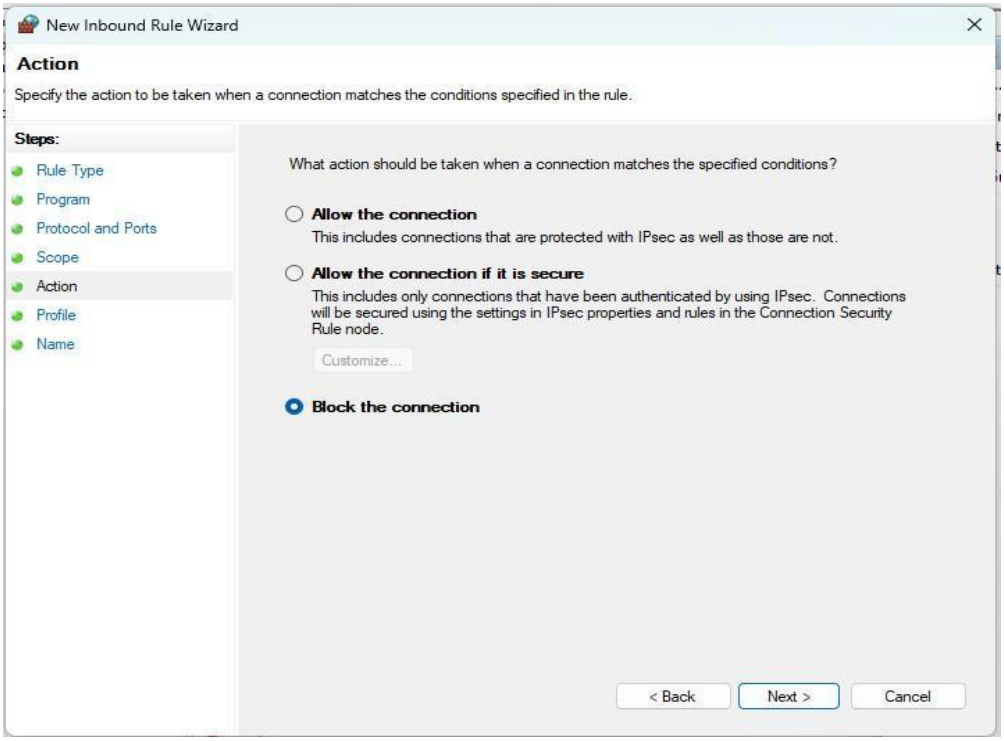


**Step 4:** Insert the IP addresses both IPv4 and IPv6.

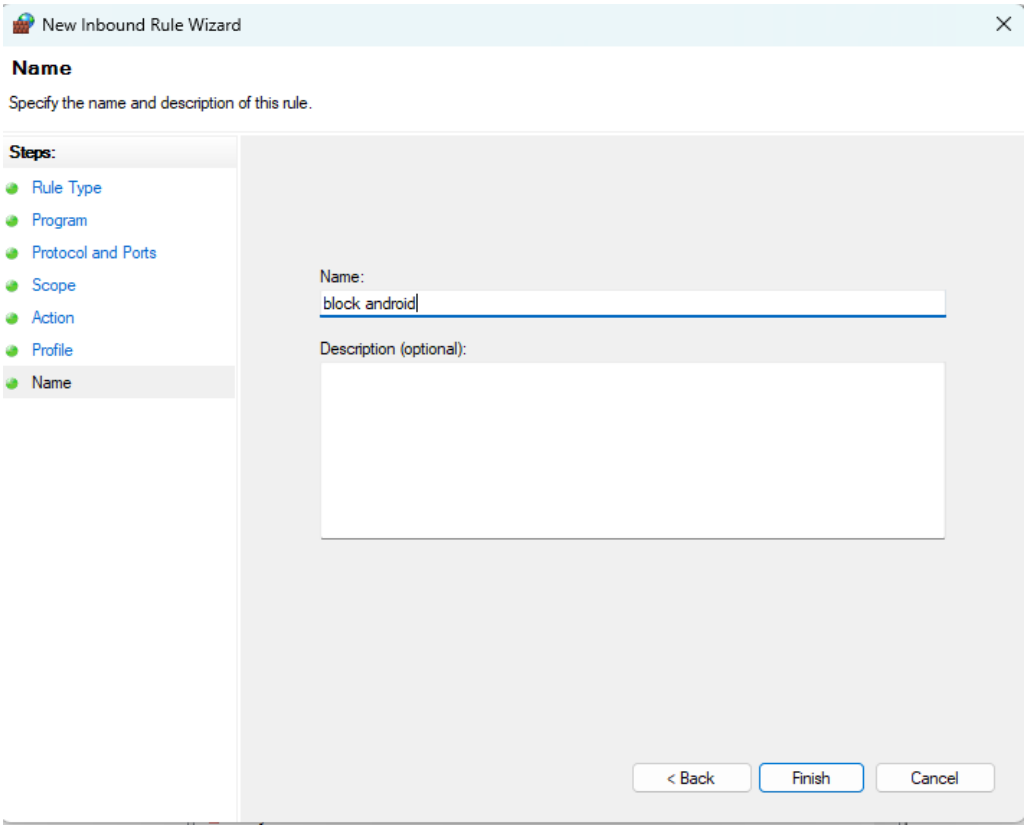


**Step 5:** Select Block connection

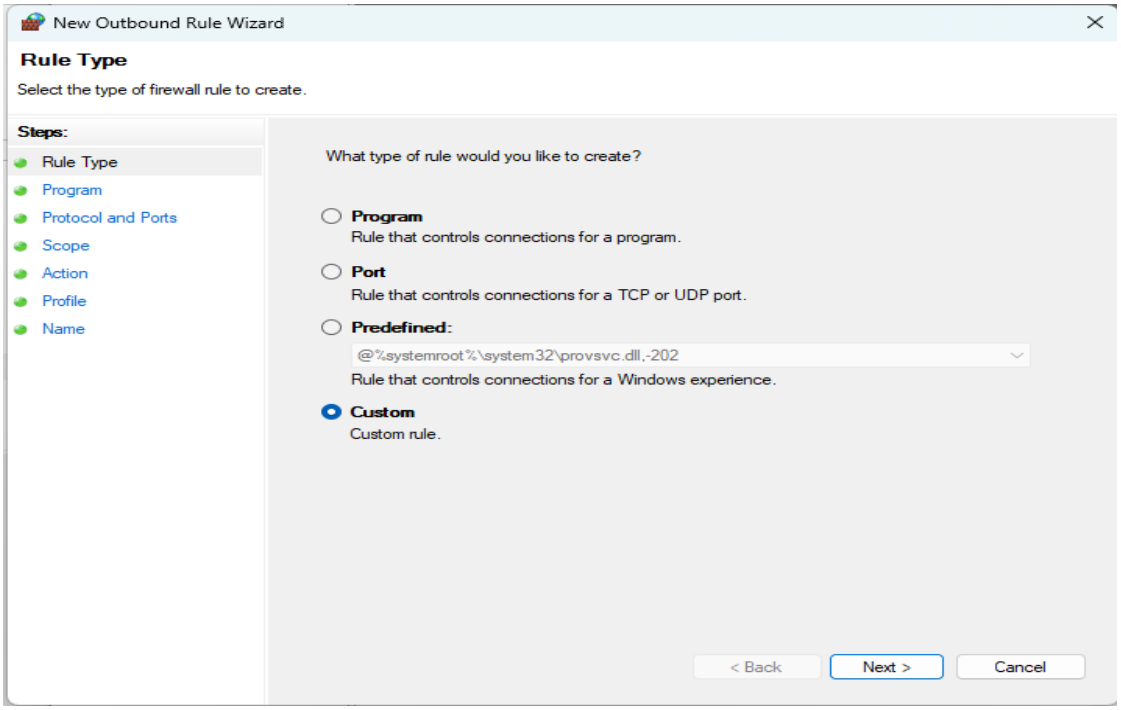



**Step 6:** Provide a suitable name and finish.

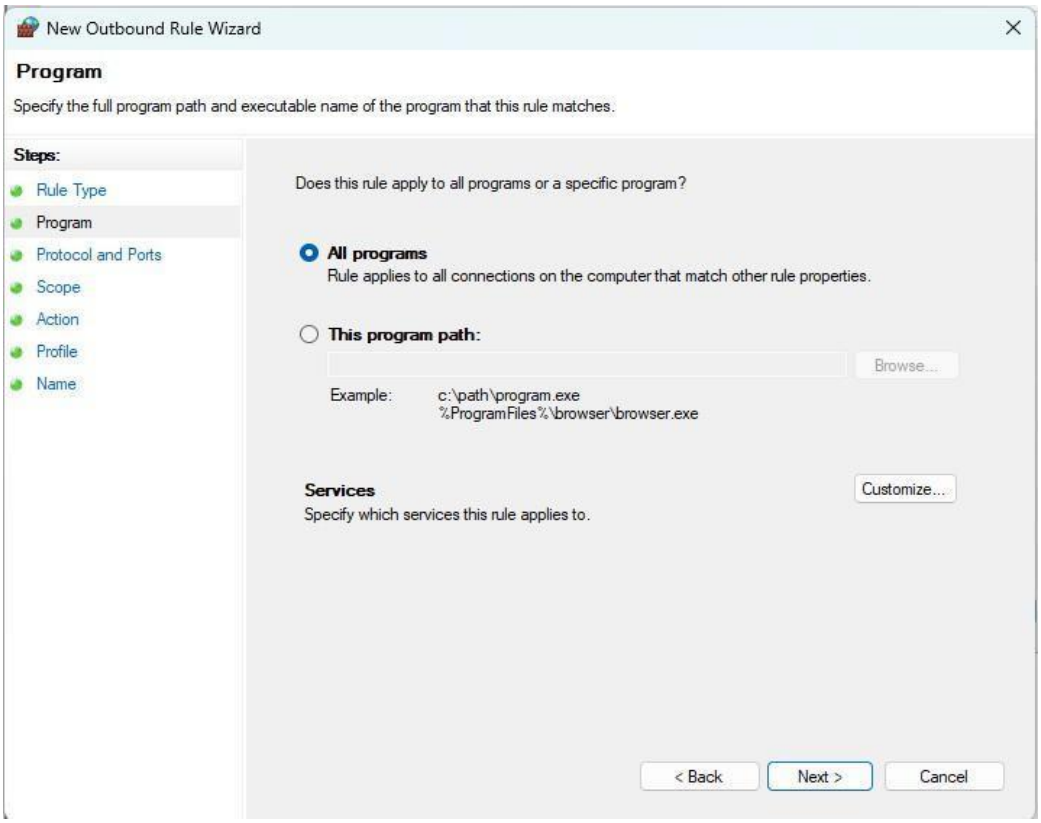


Repeat the above steps for Outbound Rules.

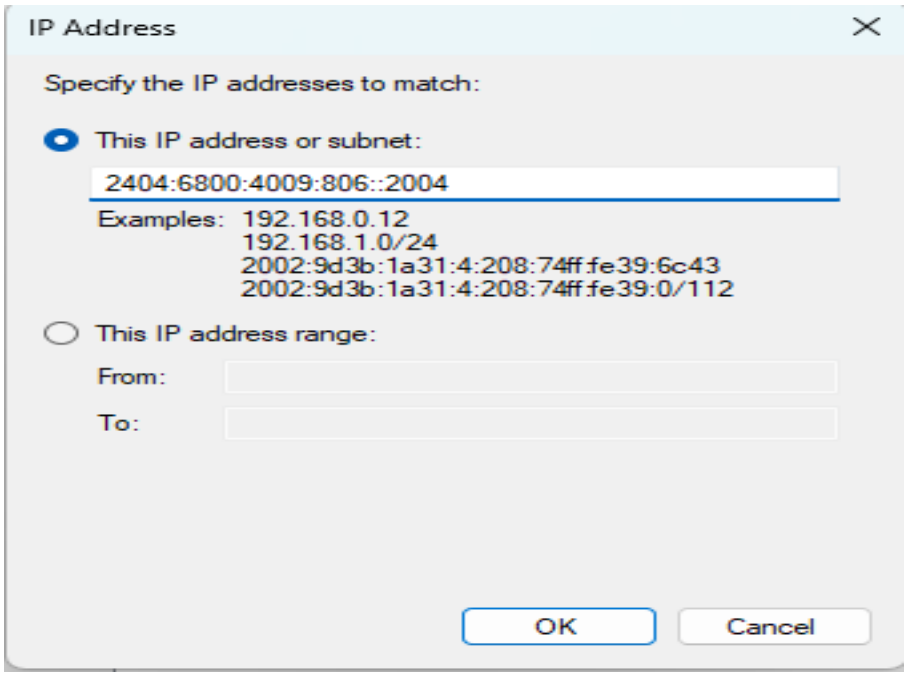
**Step 1:** We open the windows Firewall settings and apply the Inbound Rule

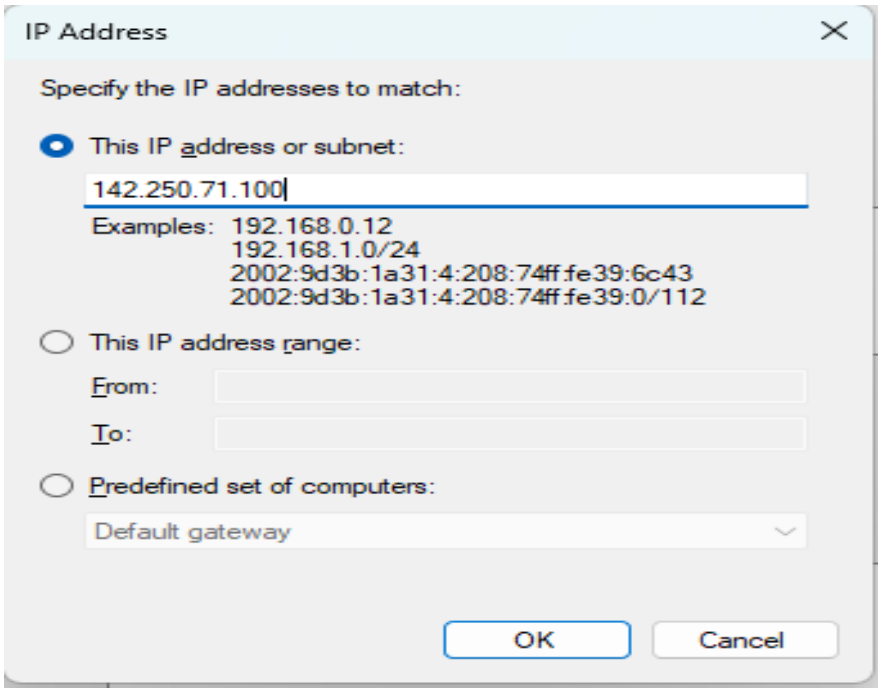



**step 2** : select all program and click next .

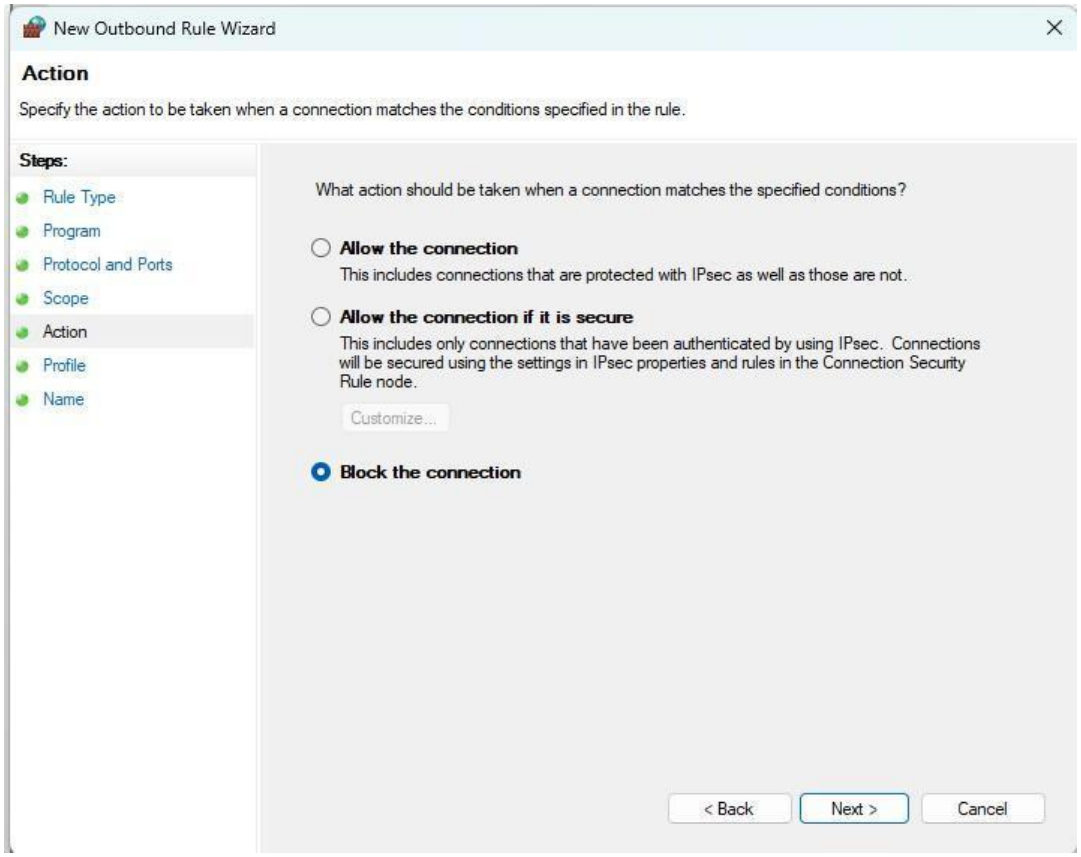


**Step 4:** Insert the IP addresses both IPv4 and IPv6.

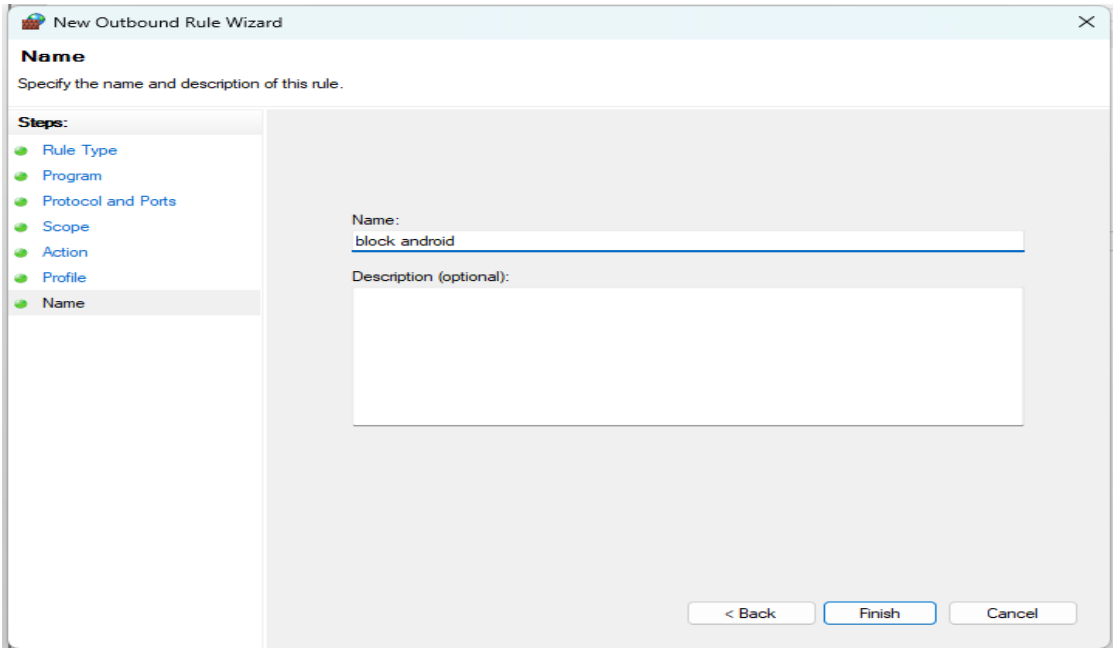


**Step 5:** Select Block connection




**Step 6:** Provide a suitable name and finish



**Output:**

Now if we try to access the website www.android.com it would be blocked .

